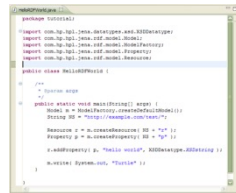


Register Allocation

Chapter 11

Where We Are

Source Code



Lexical Analysis

Parsing

Semantic Analysis

IR Generation

IR Optimization

Code Generation

Optimization

[illegible]

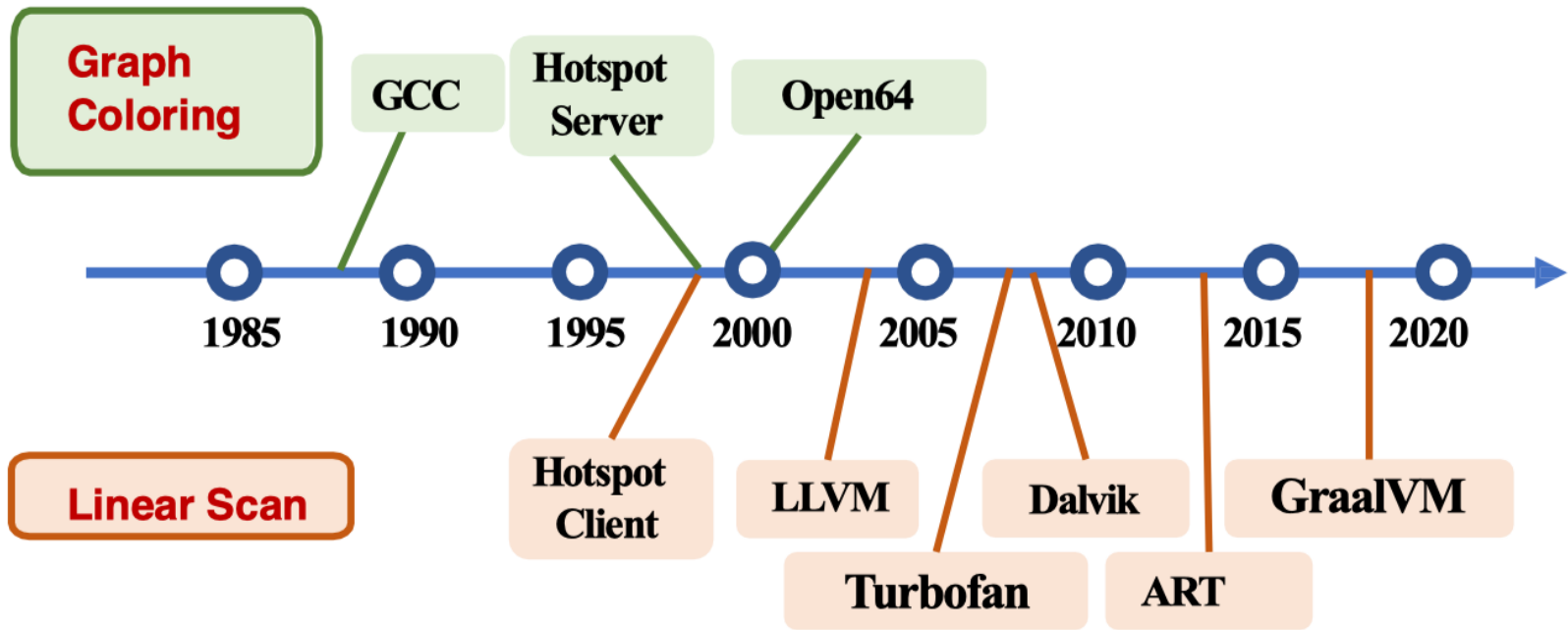
Machine Code

Register Allocation: Why & What

- Speed: Registers > Memory
 - Registers are 2x – 7x faster than cache
- Physical machines have a **limited number** of registers
- The job of **the register allocator** is to assign the *many temporaries* to *K machine registers*
- Requirements:
 - Produce correct code using k or fewer registers
 - Minimize loads, stores, and space to hold spilled values
 - Efficient register allocation

Register Allocation: How

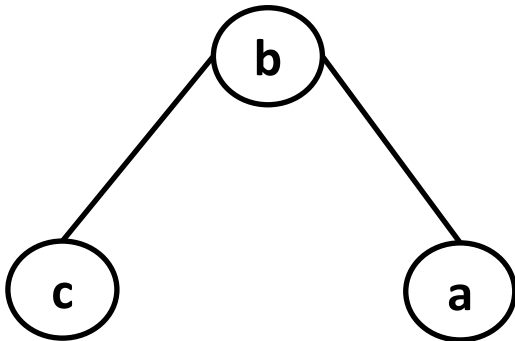
Graph Coloring: effective but less efficient



Linear Scan: more efficient, similar effectiveness

Register Allocation via Graph Coloring

- How:
 - Derive an **interference graph**
 - **Coloring** the inference graph
- No pair of nodes connected by an inference edge may be assigned the same color.
 - Colors correspond to registers



Outline

- Coloring By Simplification
- Coalescing
- Precolored Nodes

Outline

- **Coloring By Simplification**
- Coalescing
- Precolored Nodes

Coloring By Simplification

- Register allocation is an NP-complete problem
 - Graph coloring is also NP-complete
- A linear-time approximation algorithm that gives good results:
 - Its principal phases: **Build**, **Simplify**, **Spill**, and **Select**

Build

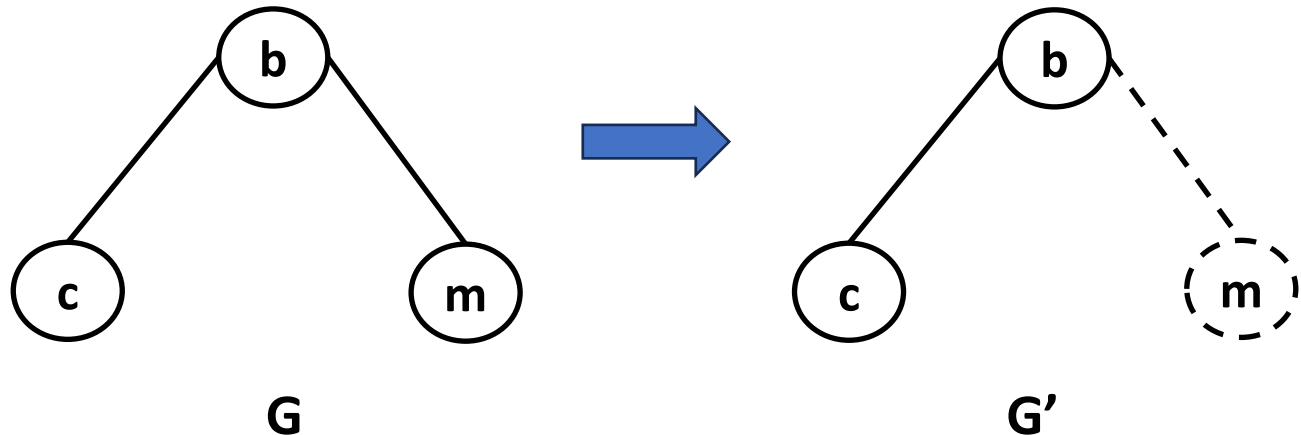
- Construct the *interference graphs*:
 - *Each node* represents a temporary value
 - *An edge* (t1, t2) indicates a pair of temporaries that cannot be assigned to the same register.
 - Analyze for all program points
- The *most common* reason for an interference edge is that *t1* and *t2* are live at the same time

Simplify

Color the graph using a simple heuristic:

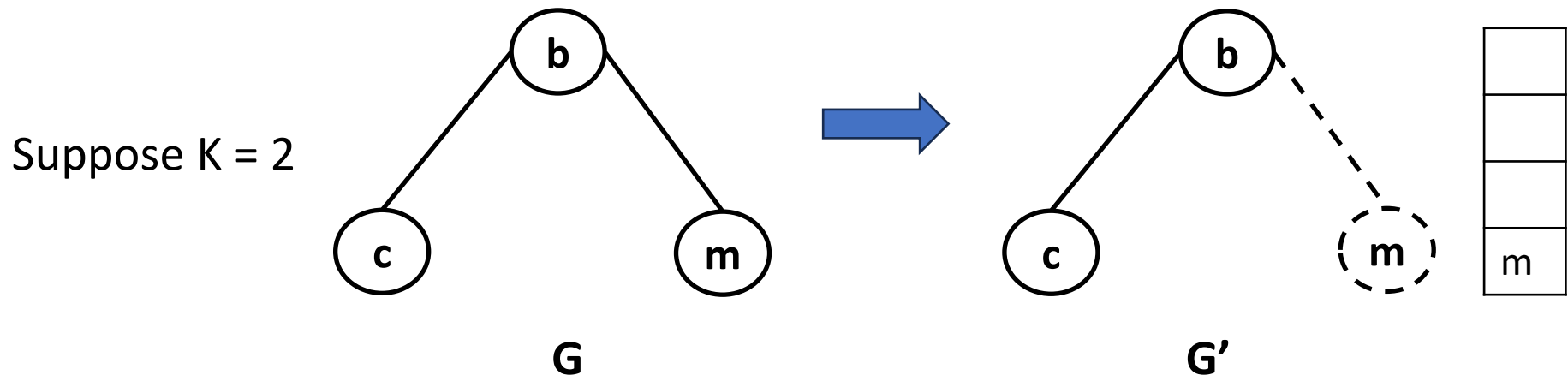
- Suppose the graph G contains a node m with fewer than K neighbors (K : the number of machine registers)
- Let G' be the graph $G - \{m\}$ obtained by removing m
- if G' can be colored, then so can G
 - why?

Suppose $K = 2$



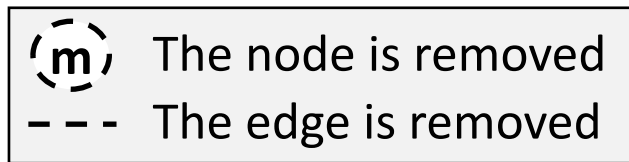
Simplify

- This lead naturally to a *stack-based (or recursive)* algorithm for coloring
 - Repeatedly **remove** (and **push** on a stack) nodes of degree less than K .
 - Each such simplification will decrease the degrees of other nodes, leading to more opportunities for simplification

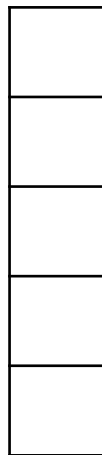


Simplify: Example

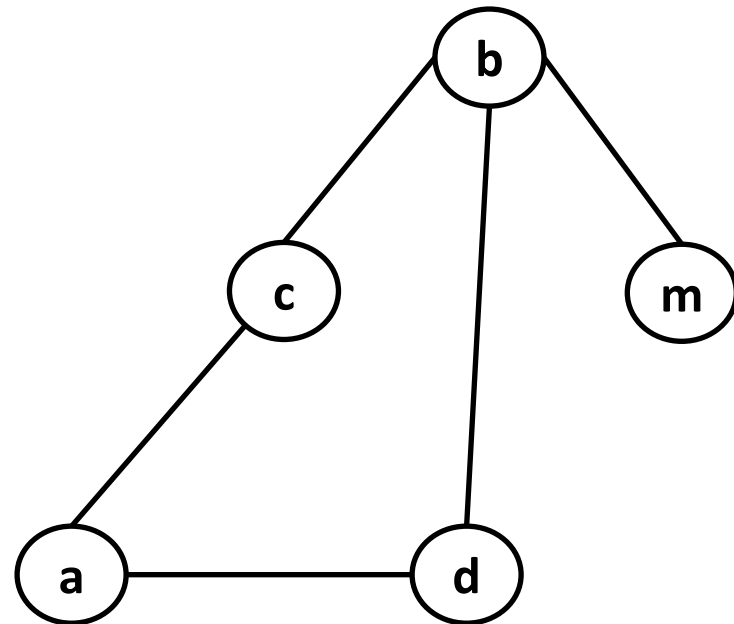
- A node with degree $< K$ is always K -colorable
- Recursively remove k -colorable nodes from the graph and push them to a stack, until the graph contains nodes only of **significant degree** (that is, nodes of $\text{degree} \geq K$)



Stack:

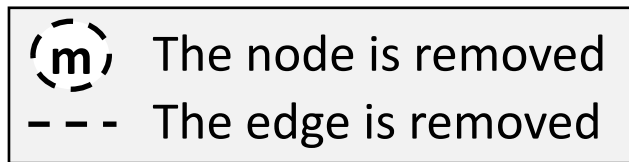


Suppose $K = 2$

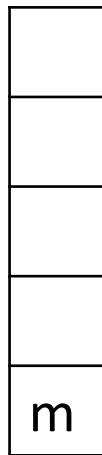


Simplify: Example

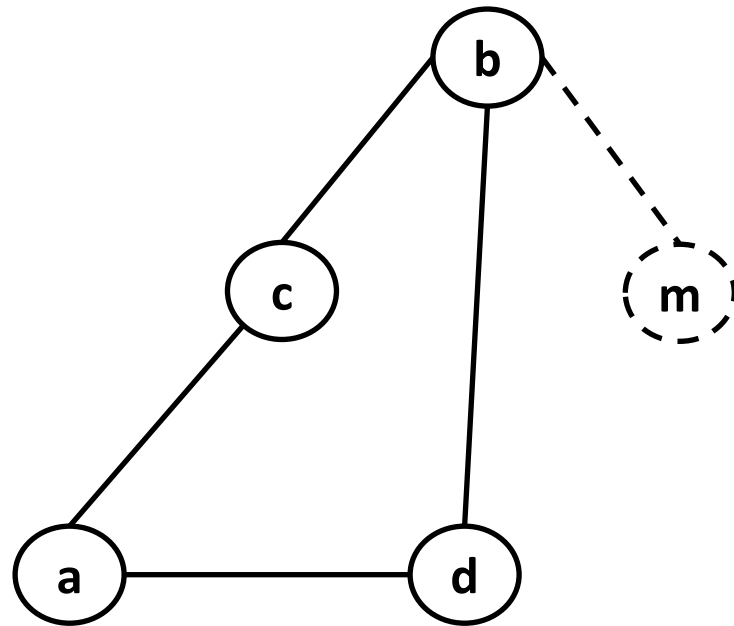
- A node with degree $< K$ is always K -colorable
- Recursively remove K -colorable nodes from the graph and push them to a stack, until the graph contains nodes only of significant degree (that is, nodes of degree $\geq K$)



Stack:



Suppose $K = 2$

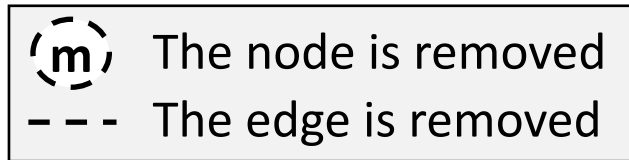


Spill

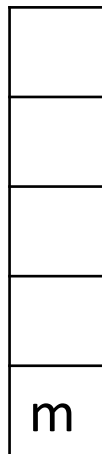
- At some point during simplification, the graph G has nodes only of **significant degree** (that is, nodes of **degree $\geq K$**).
- **Spilling**: We MAY choose some node in the graph and decide to represent it in memory, not registers
- Optimistic Coloring
 - An **optimistic approximation** to the effect of spilling: **the spilled node does not interfere with any of the other nodes remaining in the graph**
 - It can therefore be removed and pushed on the stack, and the simplify process continued.

Spill: Example

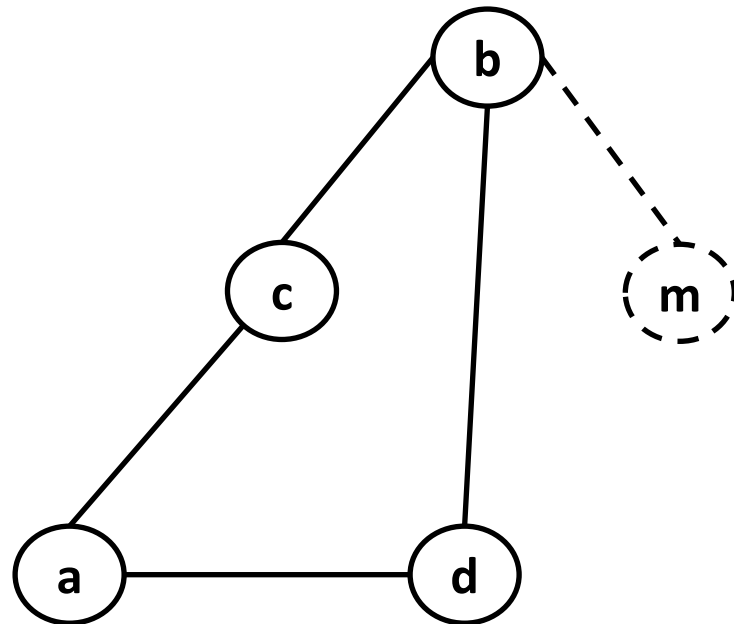
- Pick a node as a candidate for spilling
 - Remove it from the graph and put it into the stack
 - E.g., choose **b**



Stack:

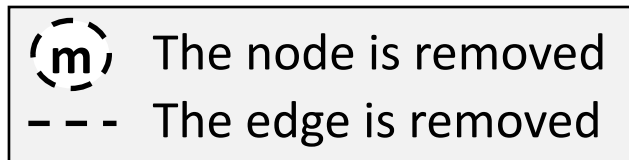


Suppose $K = 2$

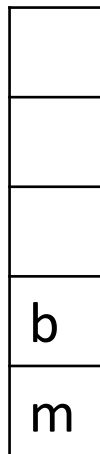


Spill: Example

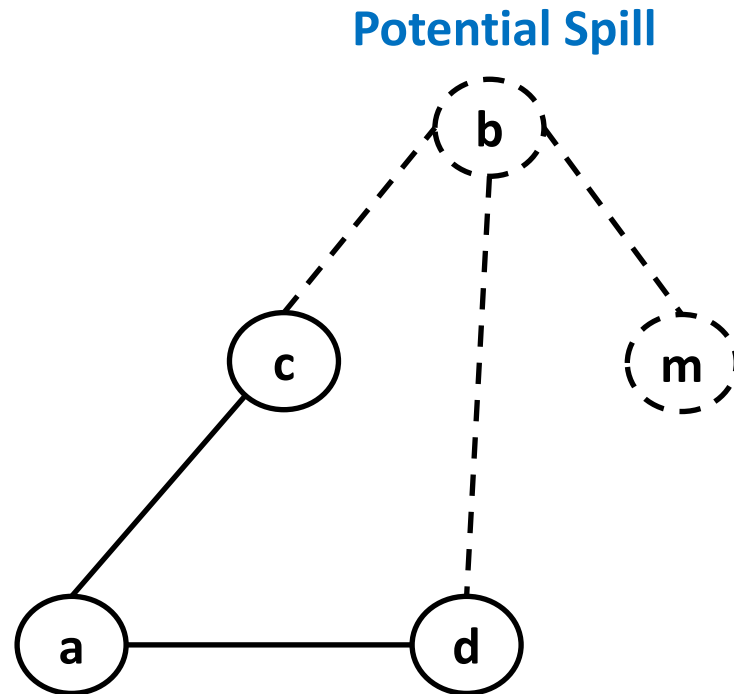
- Pick a node as a candidate for spilling
 - Remove it from the graph and put it into the stack
 - E.g., choose **b**
 - Then, continue the simplification: **d**



Stack:

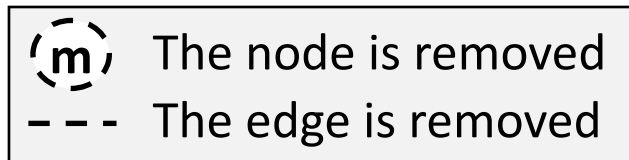


Suppose $K = 2$

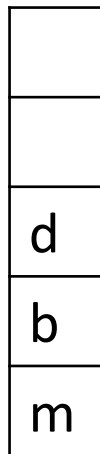


Spill: Example

- Pick a node as a candidate for spilling
 - Remove it from the graph and put it into the stack
 - E.g., choose **b**
 - Then, continue the simplification: **d**

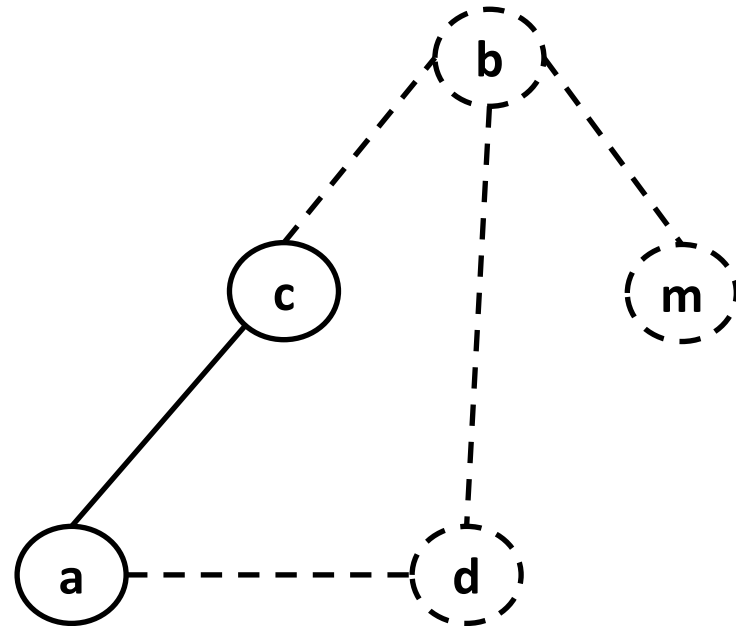


Stack:



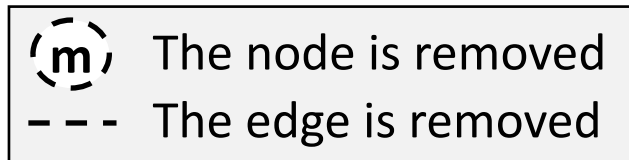
Suppose $K = 2$

Potential Spill

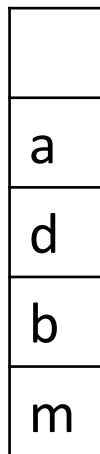


Spill: Example

- Pick a node as a candidate for spilling
 - Remove it from the graph and put it into the stack
 - E.g., choose **b**
 - Then, continue the simplification: **d**, **a**

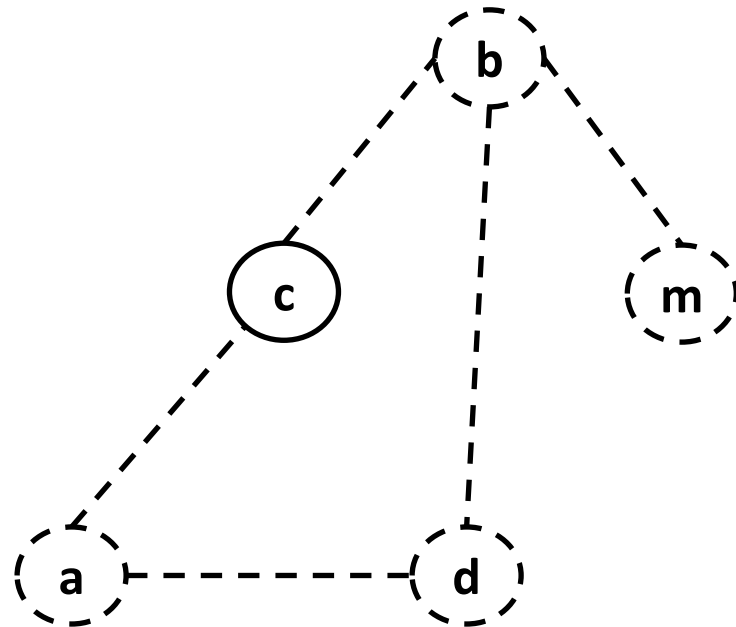


Stack:



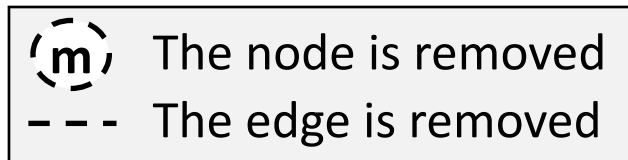
Suppose $K = 2$

Potential Spill

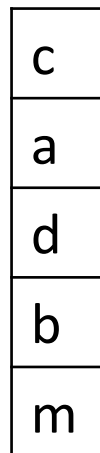


Spill: Example

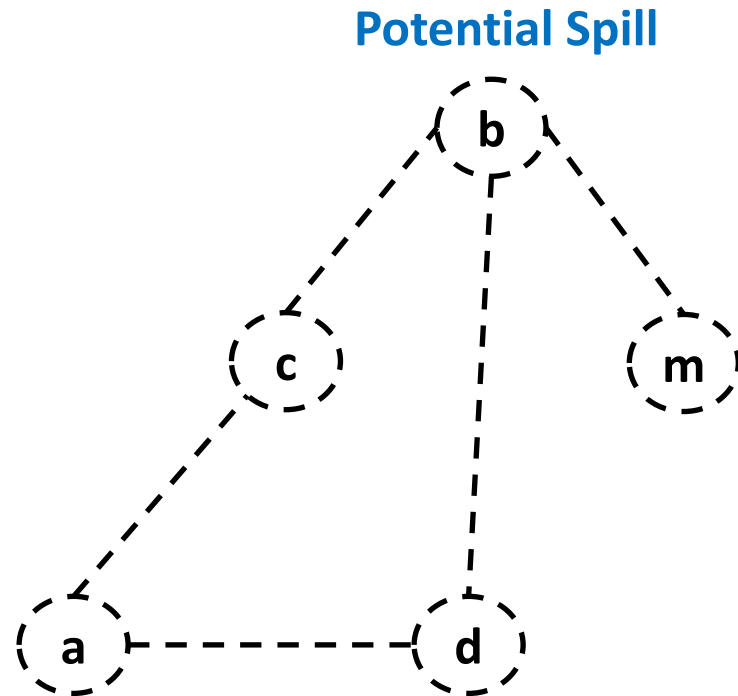
- Pick a node as a candidate for spilling
 - Remove it from the graph and put it into the stack
 - E.g., choose **b**
 - Then, continue the simplification: **d**, **a**, **c**



Stack:



Suppose $K = 2$



We may spill more than once until the graph becomes empty.

Select

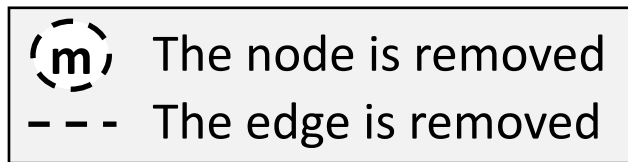
Assign colors to nodes in the graph

- Starting with the empty graph, we rebuild the original graph by repeatedly adding a node from the top of the stack.
- When we add a node to the graph, there must be a color for it.

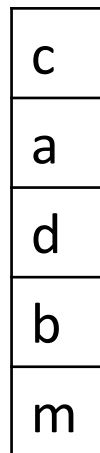
Select: Example

Assign colors to nodes in the graph

- Starting with the empty graph, we rebuild the original graph by repeatedly adding a node from the top of the stack.
- When we add a node to the graph, there must be a color for it.

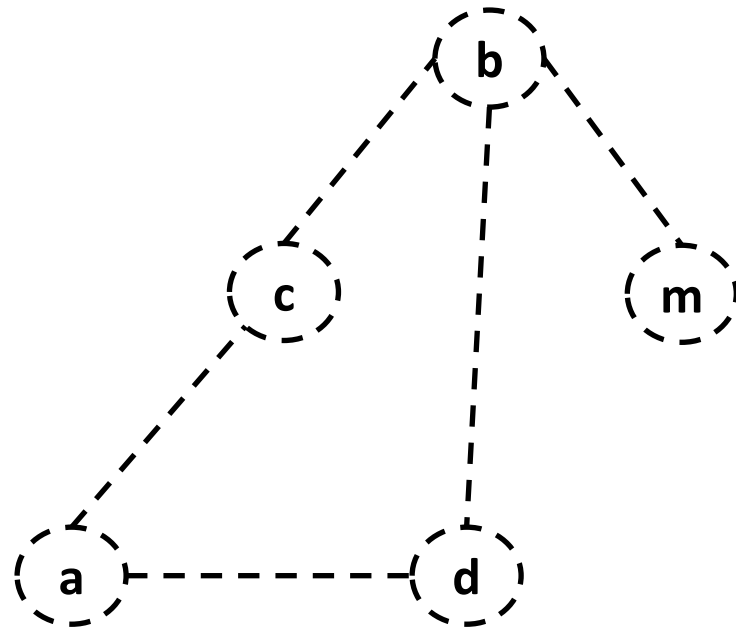


Stack:



Suppose $K = 2$

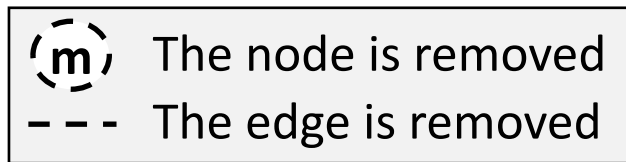
Potential Spill



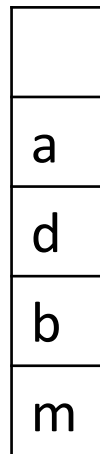
Select: Example

Assign colors to nodes in the graph

- Starting with the empty graph, we rebuild the original graph by repeatedly adding a node from the top of the stack.
- When we add a node to the graph, there must be a color for it.

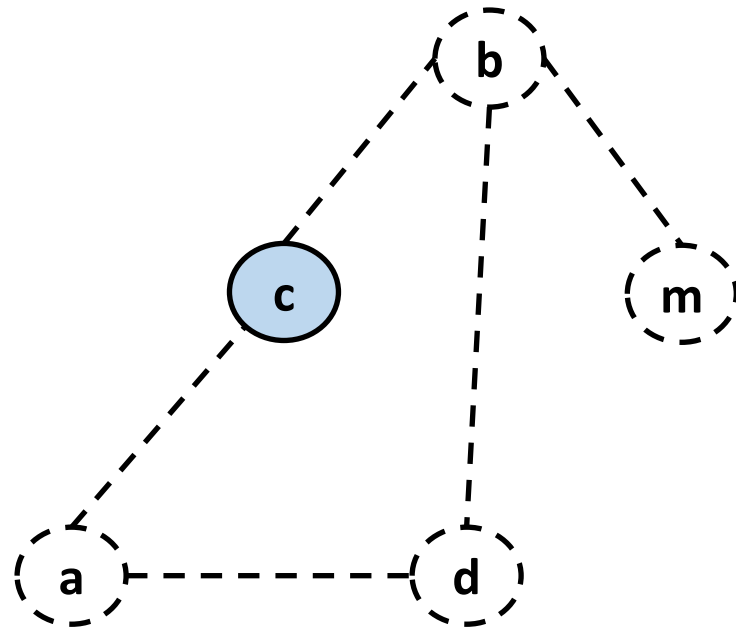


Stack:



Suppose $K = 2$

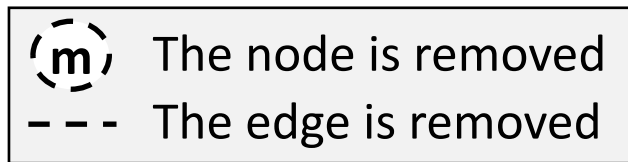
Potential Spill



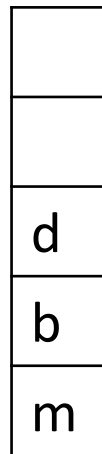
Select: Example

Assign colors to nodes in the graph

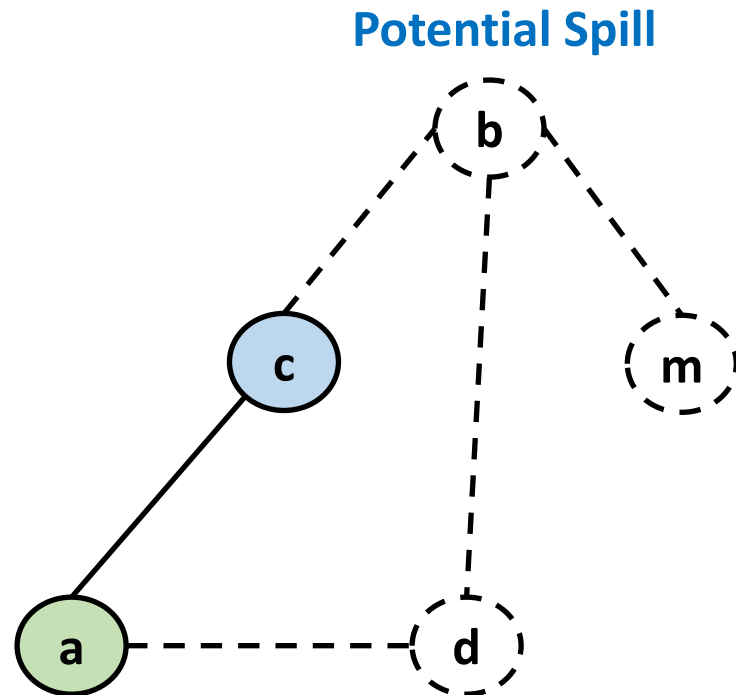
- Starting with the empty graph, we rebuild the original graph by repeatedly adding a node from the top of the stack.
- When we add a node to the graph, there must be a color for it.



Stack:



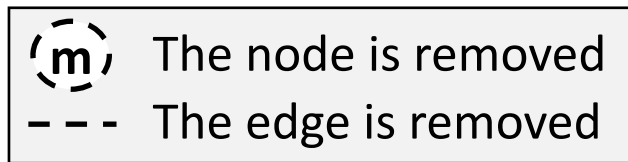
Suppose $K = 2$



Select: Example

Assign colors to nodes in the graph

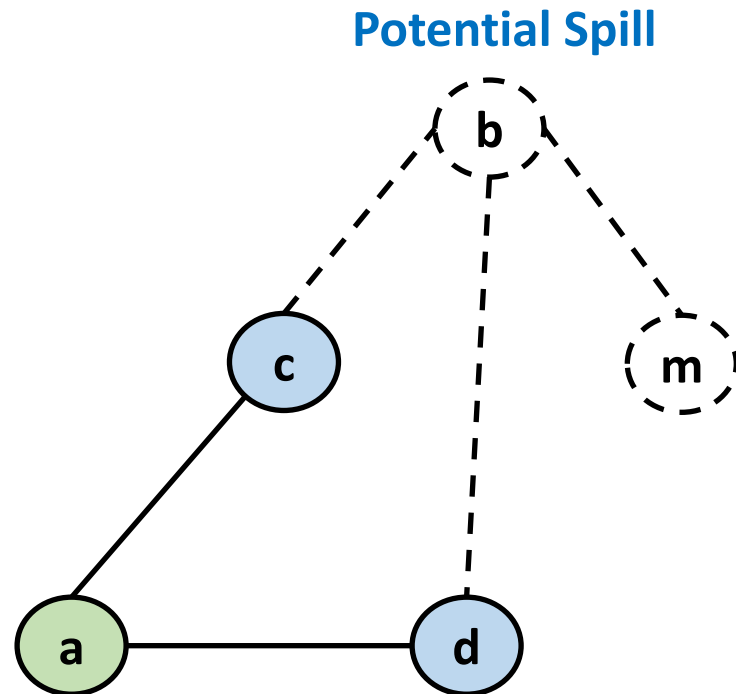
- Starting with the empty graph, we rebuild the original graph by repeatedly adding a node from the top of the stack.
- When we add a node to the graph, there must be a color for it.



Stack:



Suppose $K = 2$



Select

Assign colors to nodes in the graph

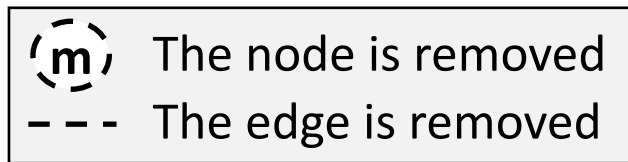
- Starting with the empty graph, we rebuild the original graph by repeatedly adding a node from the top of the stack.
- When we add a node to the graph, there must be a color for it.
- When a potential spill node **n** that was pushed using the **Spill heuristic** is popped, there is no guarantee that it will be colorable.
 - If **n**'s neighbors are colored with **fewer than K** colors, we can color **n**, and **n** does not become an actual spill.
 - This technique is known as **optimistic coloring**

c
a
d
b
m

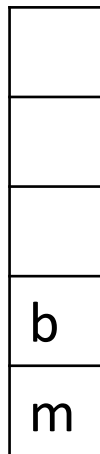
Select: Example

Assign colors to nodes in the graph

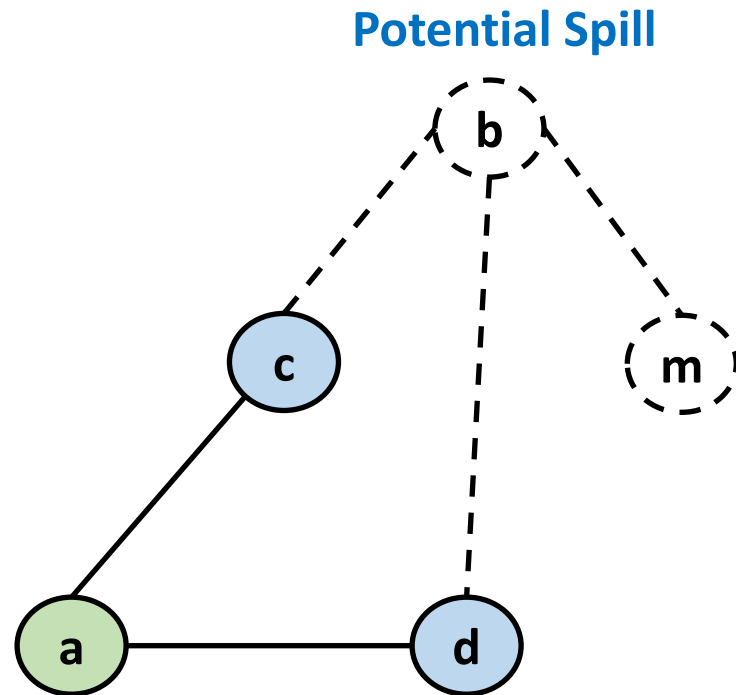
- If n 's neighbors are colored with **fewer than K** colors, we can color n , and n does not become an actual spill.
 - This technique is known as **optimistic coloring**



Stack:



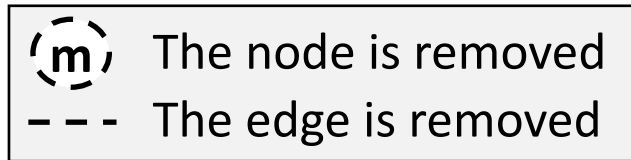
Suppose $K = 2$



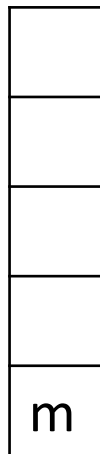
Select: Example

Assign colors to nodes in the graph

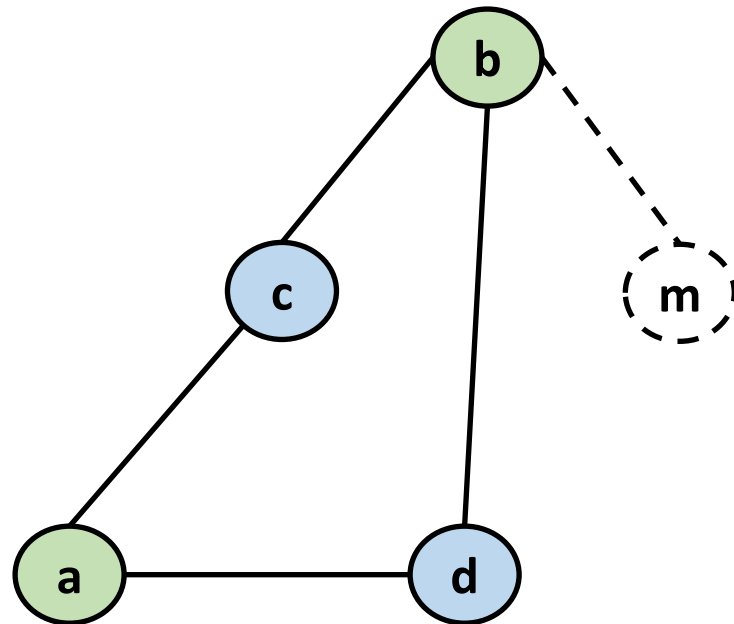
- If n 's neighbors are colored with **fewer than K** colors, we can color n , and n does not become an actual spill.
 - This technique is known as **optimistic coloring**



Stack:



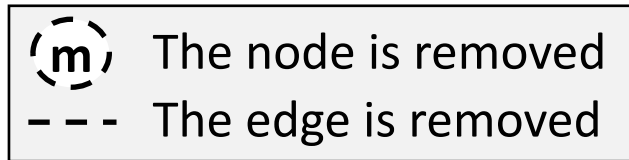
Suppose $K = 2$



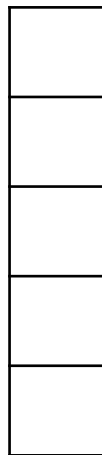
Select: Example

Assign colors to nodes in the graph

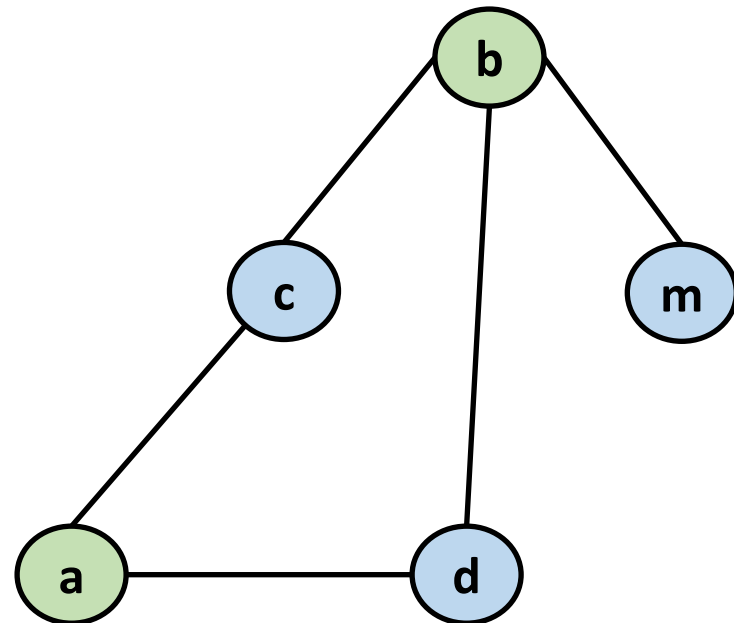
- If n 's neighbors are colored with **fewer than K** colors, we can color n , and n does not become an actual spill.
 - This technique is known as **optimistic coloring**



Stack:



Suppose $K = 2$



Select

Assign colors to nodes in the graph

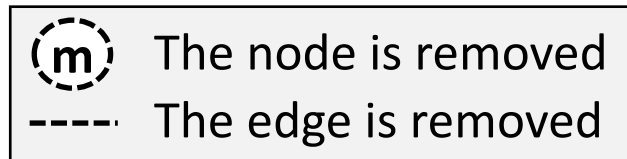
- Starting with the empty graph, we rebuild the original graph by repeatedly adding a node from the top of the stack.
- When we add a node to the graph, there must be a color for it.
- When a potential spill node **n** that was pushed using the **Spill heuristic** is popped, there is no guarantee that it will be colorable.
 - If **n**'s neighbors are colored with **fewer than K** colors, we can color **n**, and **n** does not become an actual spill.
 - This technique is known as **optimistic coloring**
 - If **n**'s neighbors have been colored with **K** different colors
 - We have an **actual spill**
 - We do not assign any color, but continue the **Select** Phase to identify other actual spills

c
a
d
b
m

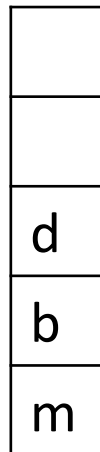
Select: Example II

Assign colors to nodes in the graph

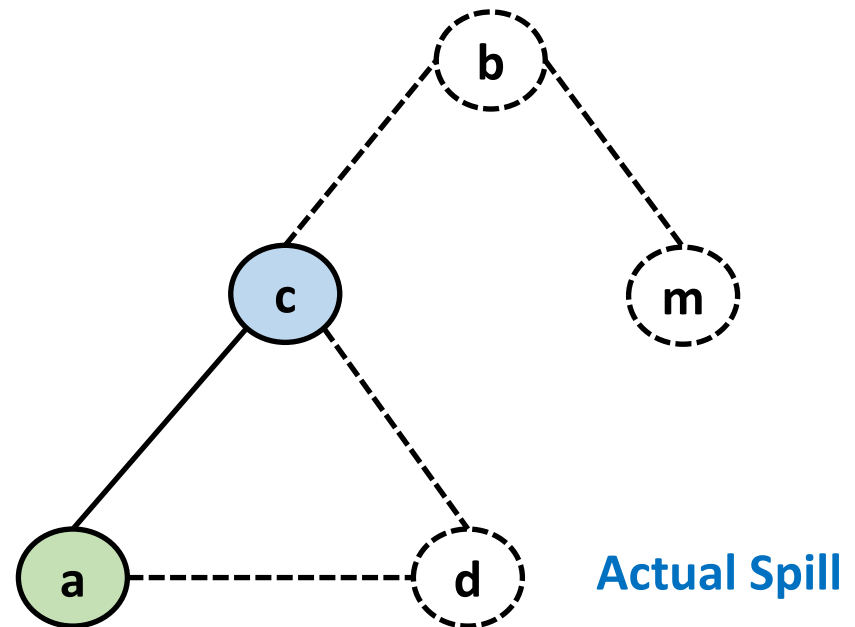
- If a potential spill n 's neighbors have been colored with K different colors
 - We have an **actual spill**. We do not assign any color, but continue the **Select** Phase to identify other actual spills



Stack:



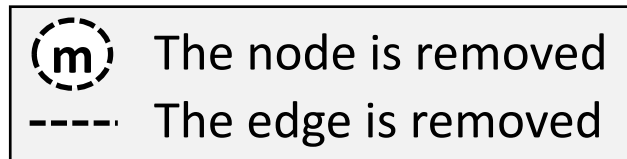
Suppose $K = 2$



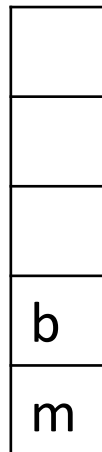
Select: Example II

Assign colors to nodes in the graph

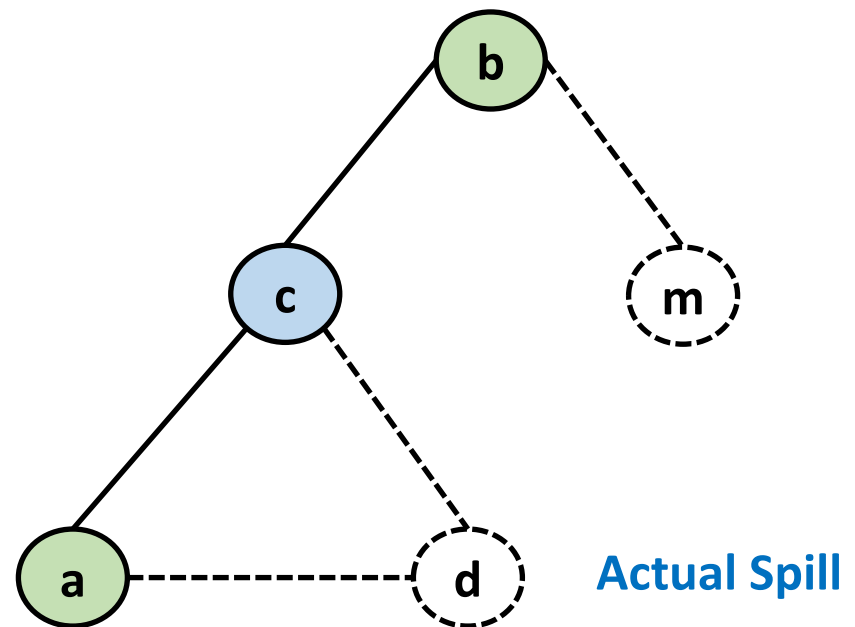
- If a potential spill n 's neighbors have been colored with K different colors
 - We have an **actual spill**. We do not assign any color, but continue the **Select** Phase to identify other actual spills



Stack:



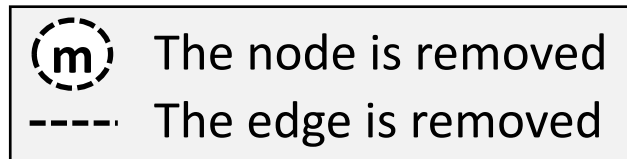
Suppose $K = 2$



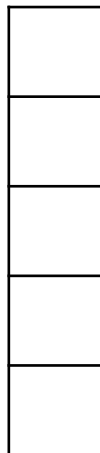
Select: Example II

Assign colors to nodes in the graph

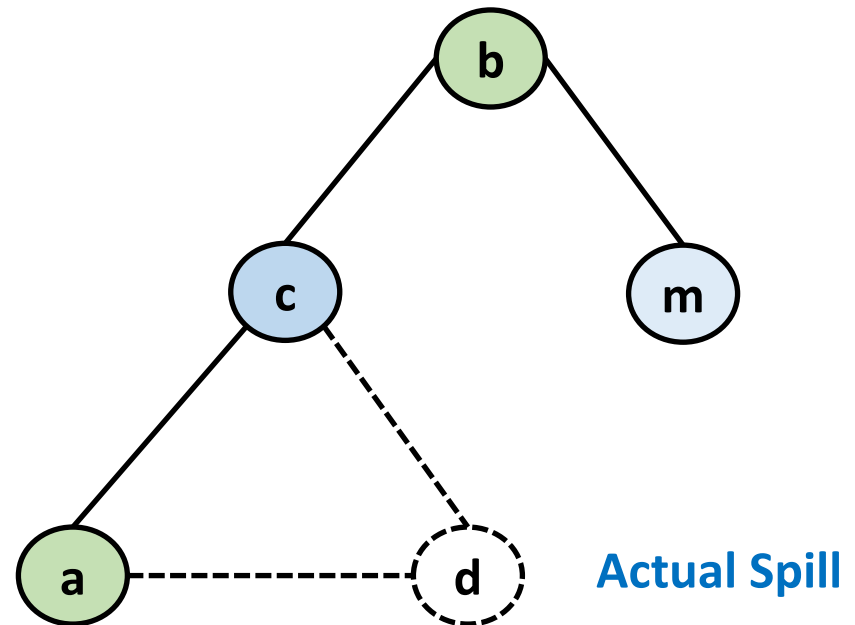
- If a potential spill n 's neighbors have been colored with K different colors
 - We have an **actual spill**. We do not assign any color, but continue the **Select** Phase to identify other actual spills



Stack:



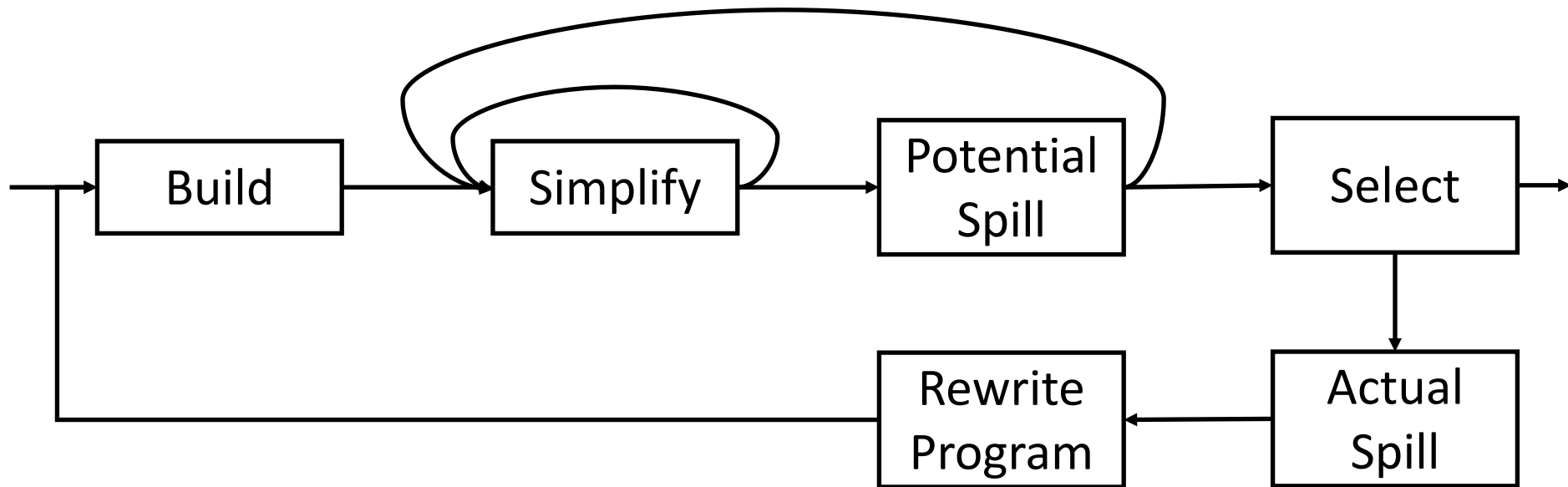
Suppose $K = 2$



Start Over

- If the **Select** phase is unable to find a color for some node(s), then the program must be **rewritten** to
 - fetch them from memory just before each **use**
 - and store them back after each **def**.
- Thus, a spilled temporary will turn into several new temporaries with tiny live ranges. These will interfere with other temporaries in the graph.
- So the algorithm is repeated on this rewritten program. This process iterates until **simplify** succeeds with no spills.
- In practice, one or two iterations almost always suffice.

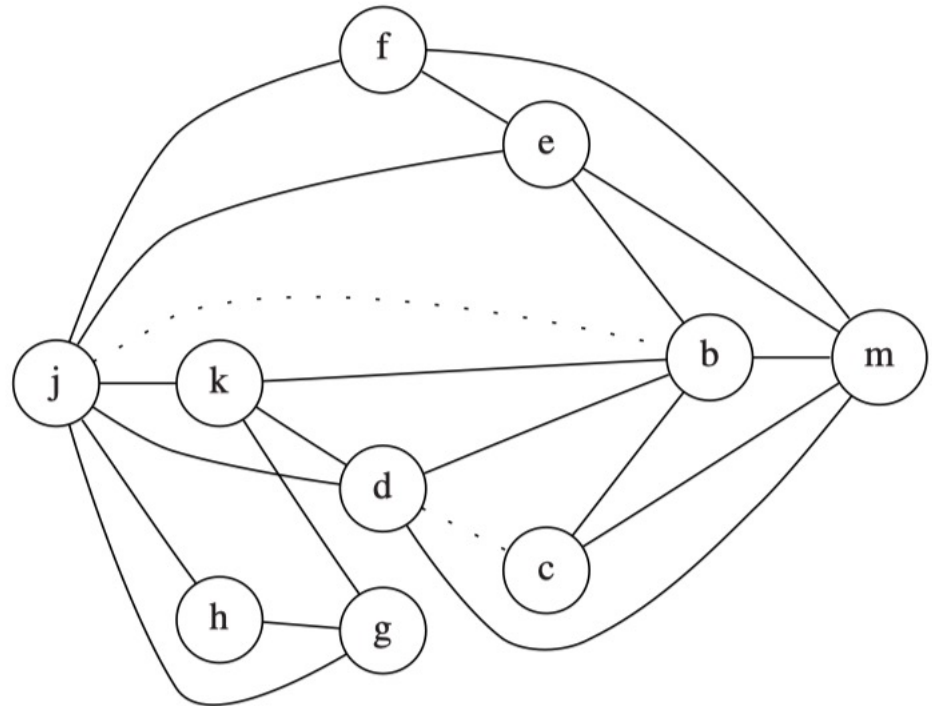
Summary



Example

- $K = 4$
- The **simplify** phase can start with the nodes **g**, **h**, **c**, and **f** in its working set.
 - Since they have less than four neighbors each

```
live-in: k j
g := mem[j+12]
h := k - 1
f := g * h
e := mem[j+8]
m := mem[j+16]
b := mem[f]
c := e + 8
d := c
k := m + 4
j := b
live-out: d k j
```

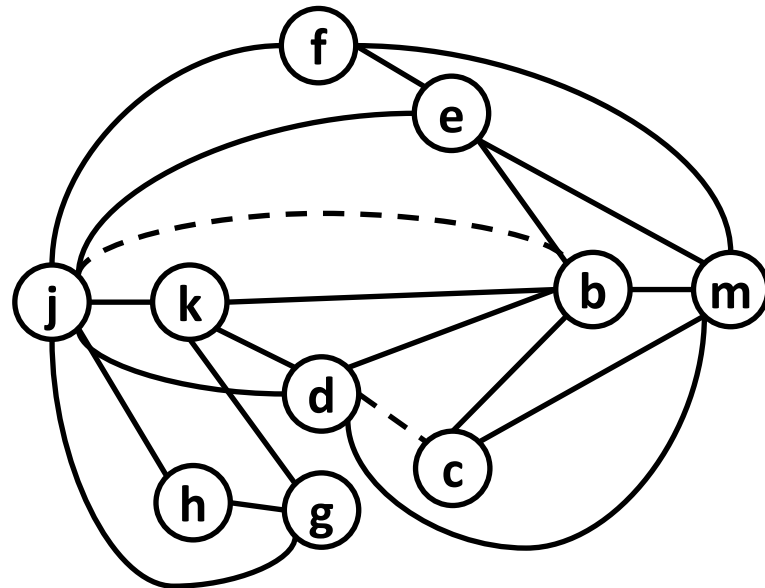


Dotted lines indicate move instructions

Example

- Removing **g** and **h** and all their edges, then node **k** becomes a candidate for removal and can be added to the work list

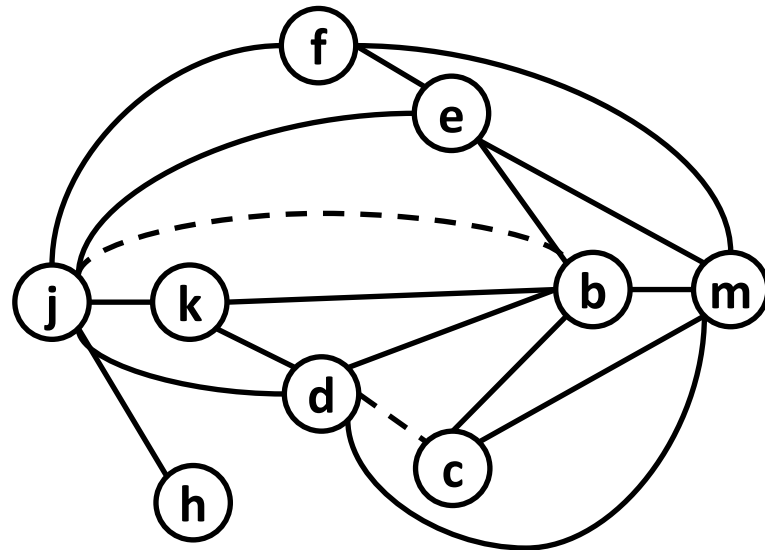
Stack	Assignment



Example

- Removing **g** and **h** and all their edges, then node **k** becomes a candidate for removal and can be added to the work list

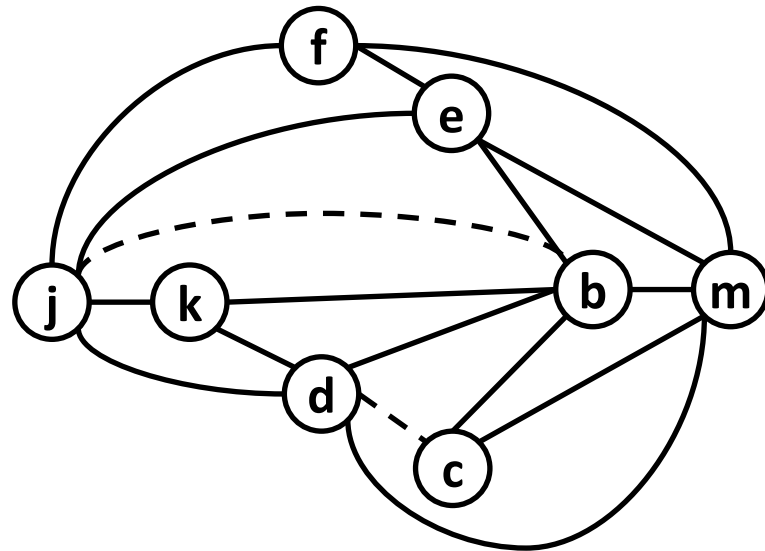
g	
Stack	Assignment



Example

- Removing **g** and **h** and all their edges, then node **k** becomes a candidate for removal and can be added to the work list

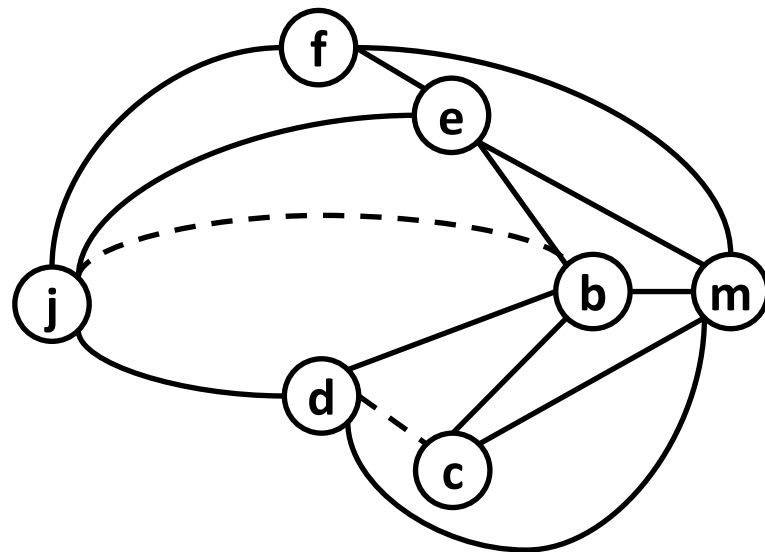
h	
g	
Stack	Assignment



Example

- Removing **g** and **h** and all their edges, then node **k** becomes a candidate for removal and can be added to the work list

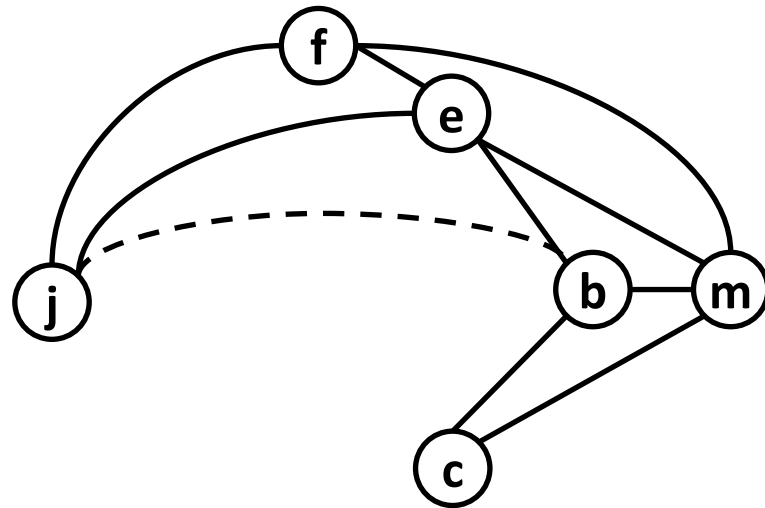
k	
h	
g	
Stack	Assignment



Example

- Remove **d**

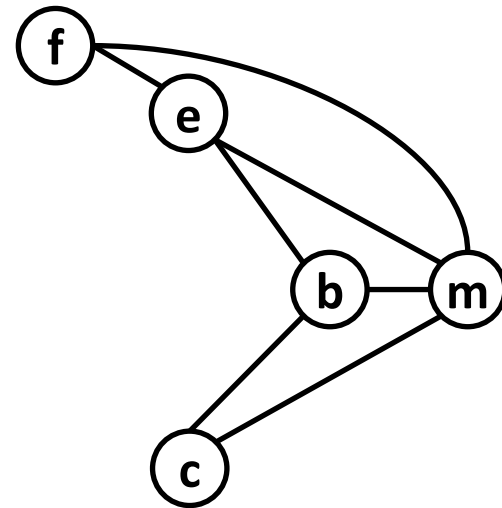
d	
k	
h	
g	
Stack	Assignment



Example

- Remove j

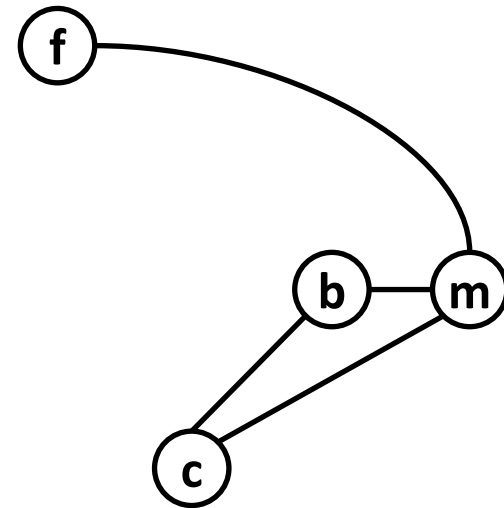
j	
d	
k	
h	
g	
Stack	Assignment



Example

- Remove **e**

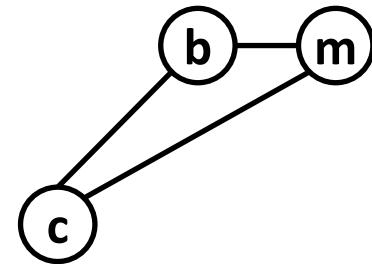
e	
j	
d	
k	
h	
g	
Stack	Assignment



Example

- Remove **f**

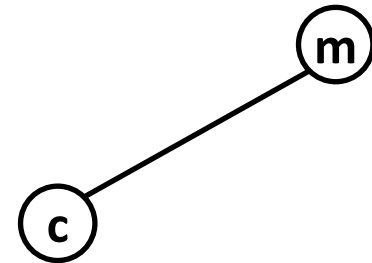
f	
e	
j	
d	
k	
h	
g	
Stack	Assignment



Example

- Remove **b**

b	
f	
e	
j	
d	
k	
h	
g	
Stack	Assignment



Example

- Remove c

c	
b	
f	
e	
j	
d	
k	
h	
g	
Stack	Assignment

m

Example

- Remove **m**

m	
c	
b	
f	
e	
j	
d	
k	
h	
g	
Stack	Assignment

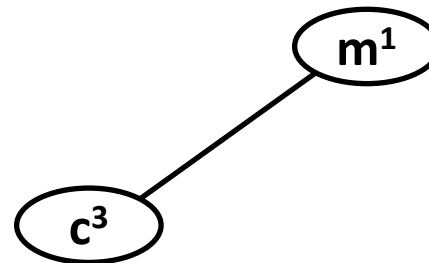
Example

m	1
c	
b	
f	
e	
j	
d	
k	
h	
g	
Stack	Assignment

m^1

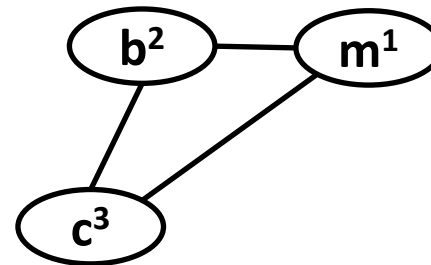
Example

m	1
c	3
b	
f	
e	
j	
d	
k	
h	
g	
Stack	Assignment



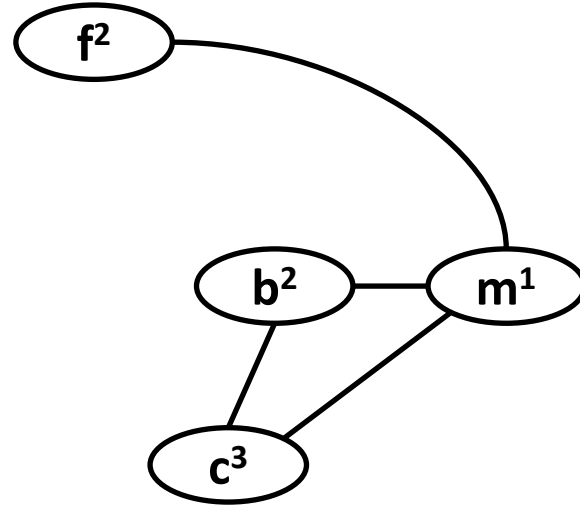
Example

m	1
c	3
b	2
f	
e	
j	
d	
k	
h	
g	
Stack	Assignment



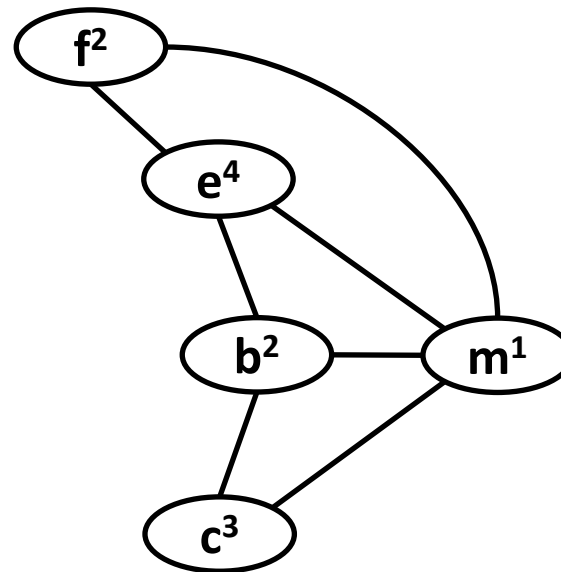
Example

m	1
c	3
b	2
f	2
e	
j	
d	
k	
h	
g	
Stack	Assignment



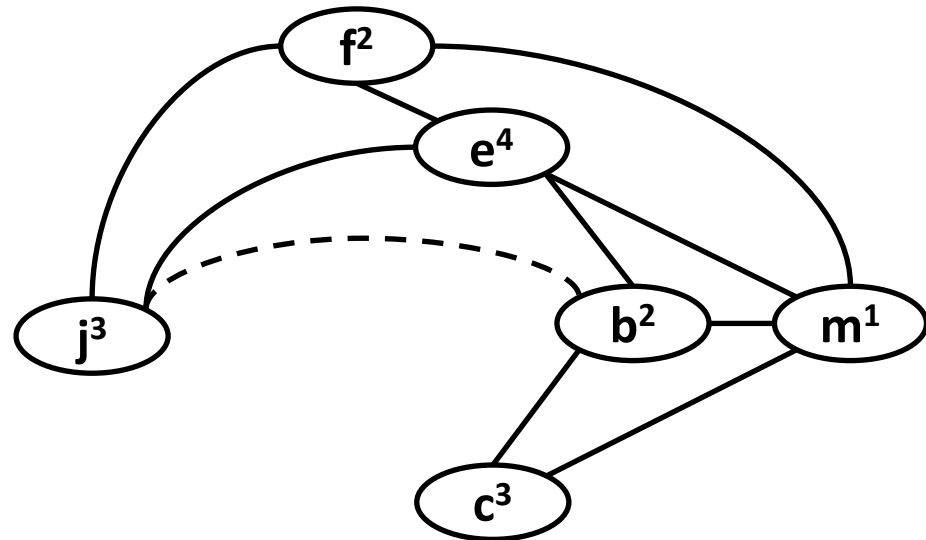
Example

m	1
c	3
b	2
f	2
e	4
j	
d	
k	
h	
g	
Stack	Assignment



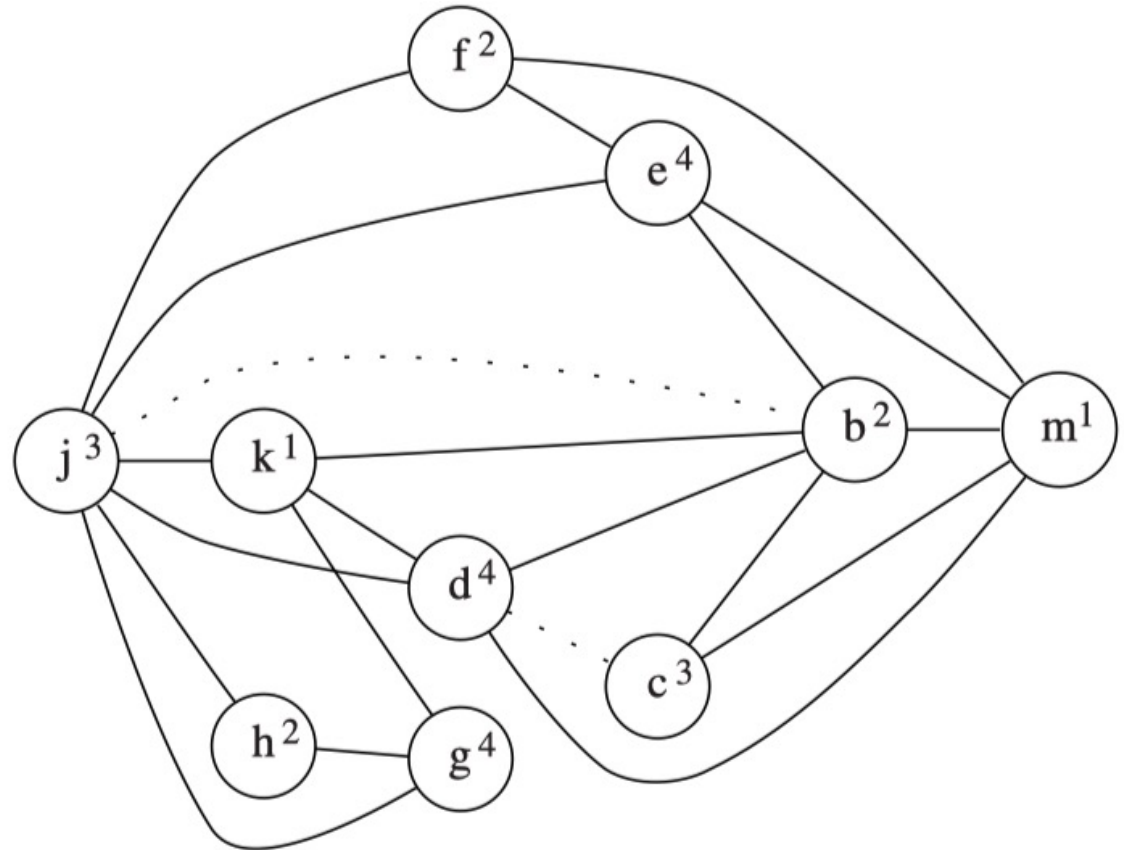
Example

m	1
c	3
b	2
f	2
e	4
j	3
d	
k	
h	
g	
Stack	Assignment



Example

m	1
c	3
b	2
f	2
e	4
j	3
d	4
k	1
h	2
g	4
Stack	Assignment



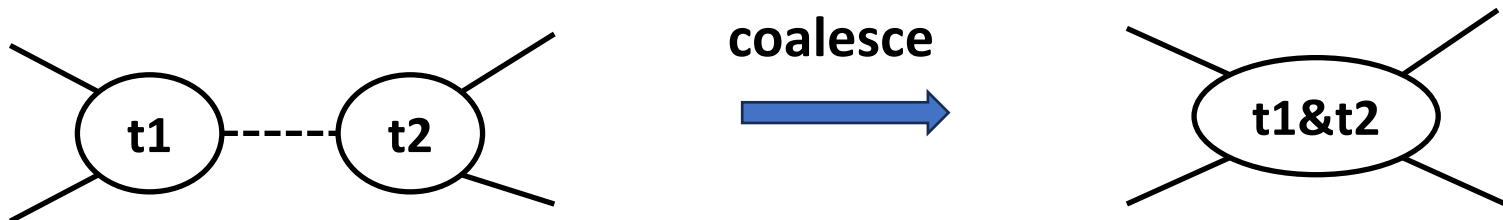
Outline

- Coloring By Simplification
- **Coalescing**
- Precolored Nodes

Coalescing: What

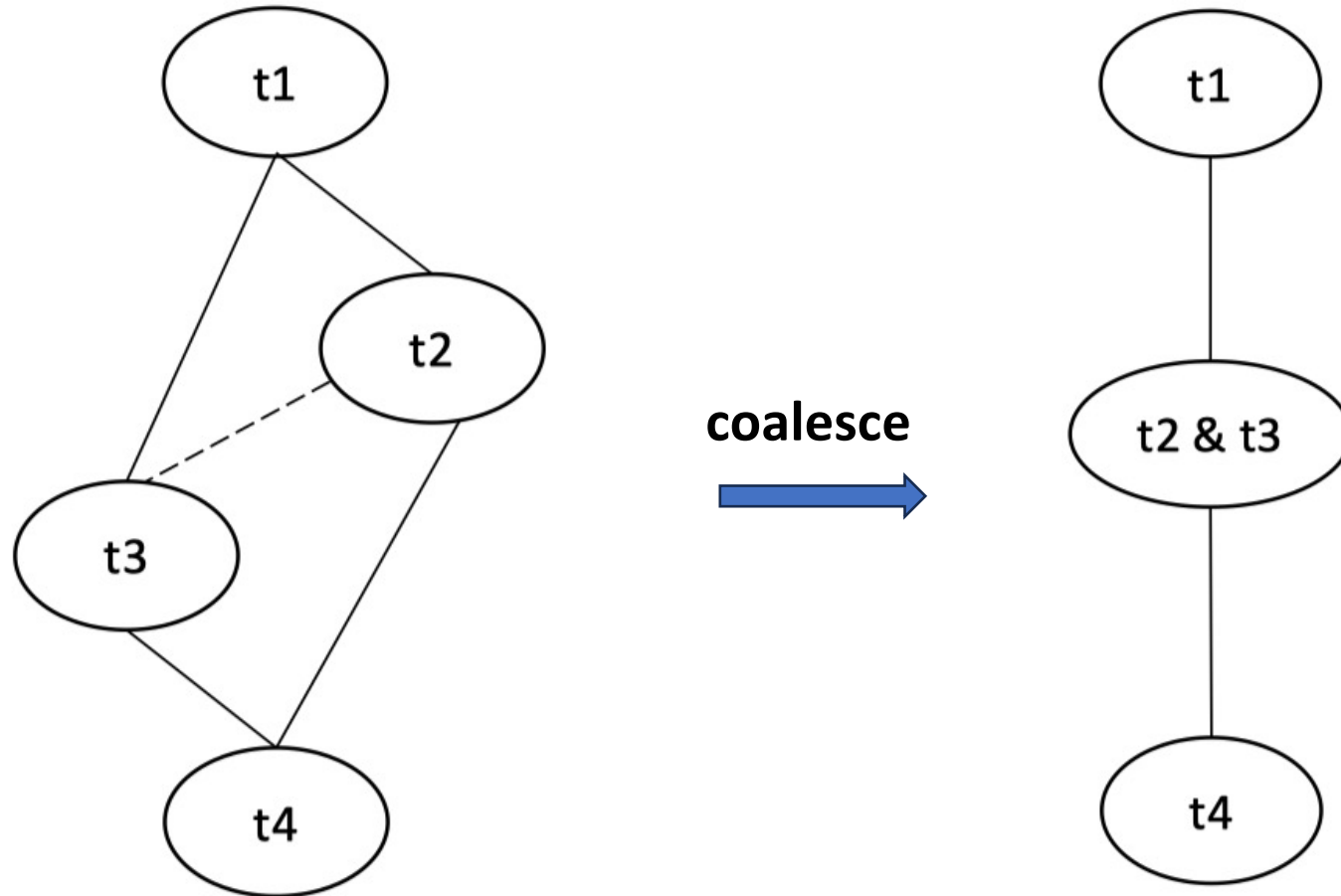
- If there is no edge in the interference graph between the source and destination of a **MOVE** instruction
 - The **MOVE** can be eliminated.
- The source and destination nodes are **coalesced** into a new node whose edges are the union of those of the nodes being replaced.

MOVE t1 t2



Coalescing: Why

- Coalescing may improve the colorability



After coalescing, t1 and t4 have one neighbor less.

Coalescing

- **Problem:**

- The node being introduced is more constrained than those being removed, as it contains a union of edges.
- A k -colorable graph before coalescing may no longer be K -colorable after reckless coalescing.

- **Idea:** conservative coalescing:

- Coalesce only where it is safe to do so.
- **safe:** The coalescing will not render the graph uncolorable

- **How** to decide whether the coalescing is safe or not?

- **Briggs**
- **George**

Briggs

- Nodes **a** and **b** can be coalesced if the resulting node **ab** will have fewer than K neighbors of significant degree (i.e., having $\geq K$ edges)
- The coalescing is guaranteed not to turn a K -colorable graph into a non- K -colorable graph.
 - **Why?**
 - The **simplify** phase has removed all the insignificant-degree nodes from the graph.
 - The coalesced node will be adjacent only to those neighbors that were of significant degree.
 - fewer than K neighbors of significant degree $\Rightarrow$ **simplify** can remove the coalesced node from the graph.

George

- Nodes **a** and **b** can be coalesced if for every neighbor **t** of **a**, either **t** already interferes with **b** or **t** is of insignificant degree ($< K$).
- This coalescing is safe.
 - Why?
 - If **t** already interferes with **b**, **(a, t)** and **(b, t)** will be merged into **(ab, t)**, not leading to the increase of degree.
 - If **t** is of insignificant degree, **t** will be removed by the **simplify** phase, also not leading to the increase of degree.

Coloring with Coalescing

How does coloring with coalescing work?

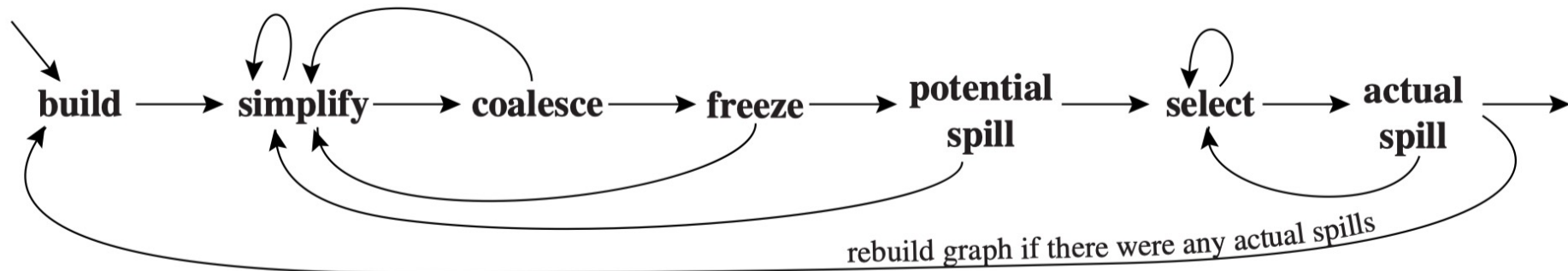


FIGURE 11.4. Graph coloring with coalescing.

- The **coalesce**, **simplify**, and **spill** procedures should be alternated until the graph is empty

Coloring with Coalescing

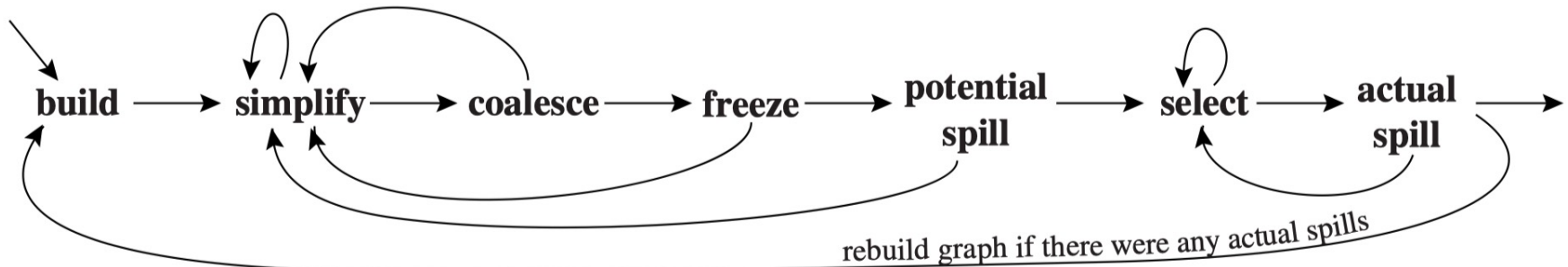


FIGURE 11.4. Graph coloring with coalescing.

1. Build

- Construct the interference graph
- Categorize each node as either **move-related** or **non-move-related**
 - A move-related node is one that is either the source or destination of a move instruction

Coloring with Coalescing

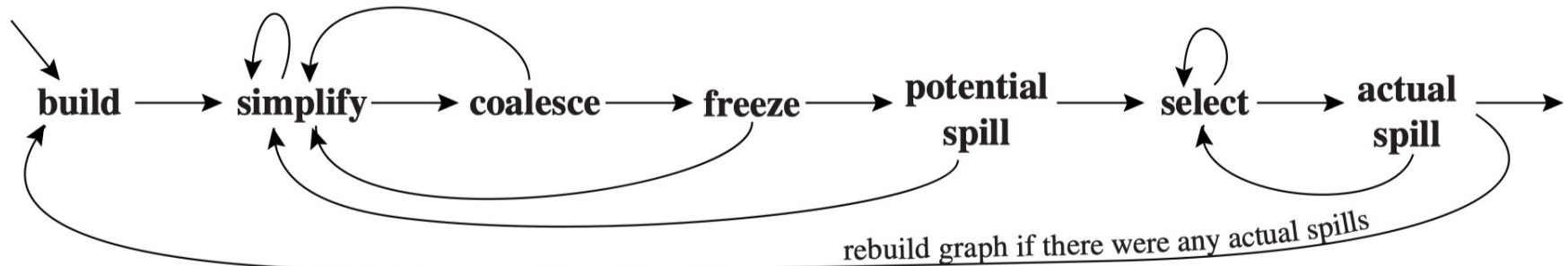


FIGURE 11.4. Graph coloring with coalescing.

2. Simplify

- One at a time, remove **non-move-related** nodes of low ($<K$) degree from the graph.

Coloring with Coalescing

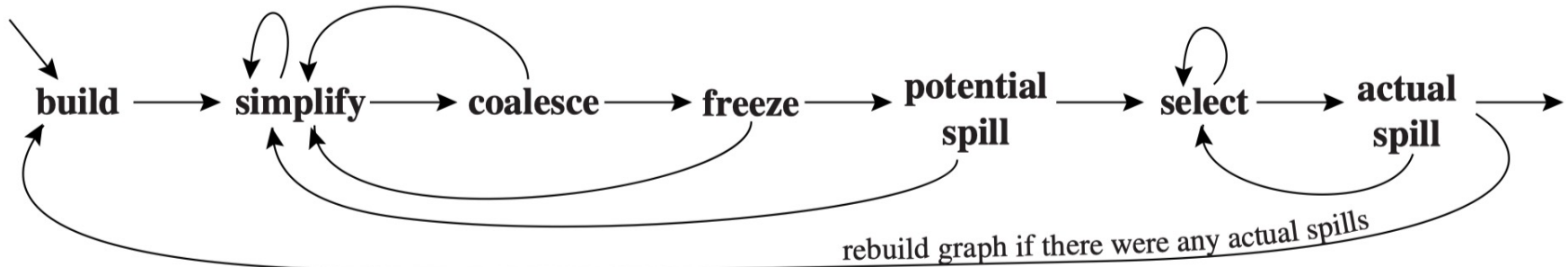


FIGURE 11.4. Graph coloring with coalescing.

3. Coalesce

- Perform conservative coalescing on the reduced graph.
- If the resulting node is no longer move-related, it will be available for the next round of **simplification**.
- **Simplify** and **coalesce** are repeated until only significant-degree or move-related nodes remain.

Coloring with Coalescing

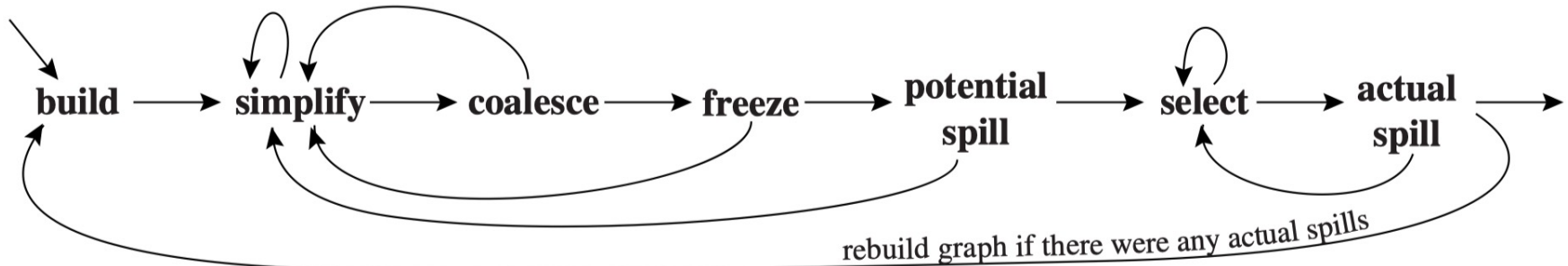


FIGURE 11.4. Graph coloring with coalescing.

4. Freeze

- If neither **simplify** nor **coalesce** applies, we look for a move-related node of low degree. We freeze the *moves* in which this node is involved.
 - We give up the hope of coalescing those moves.
 - This causes this node (and perhaps other nodes related to the frozen moves) to be considered non-move-related.
- **Simplify** and **coalesce** are resumed.

Coloring with Coalescing

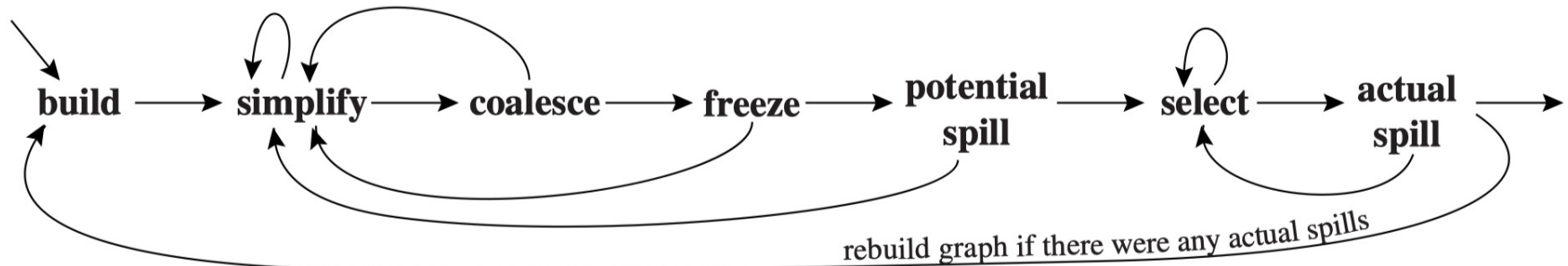


FIGURE 11.4. Graph coloring with coalescing.

5. Spill

- If there are no low-degree nodes, we select a significant-degree node for potential spilling and push it on the stack.

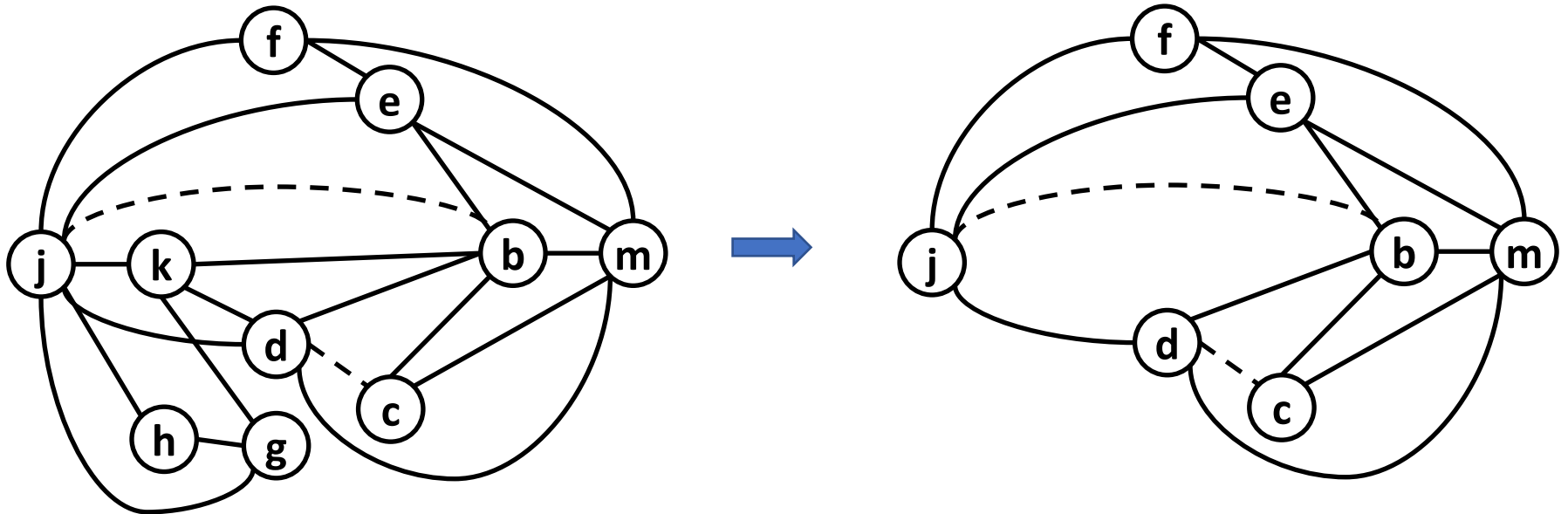
6. Select

- Pop the entire stack, assigning colors.

Rebuild the graph if there are any actual pills.

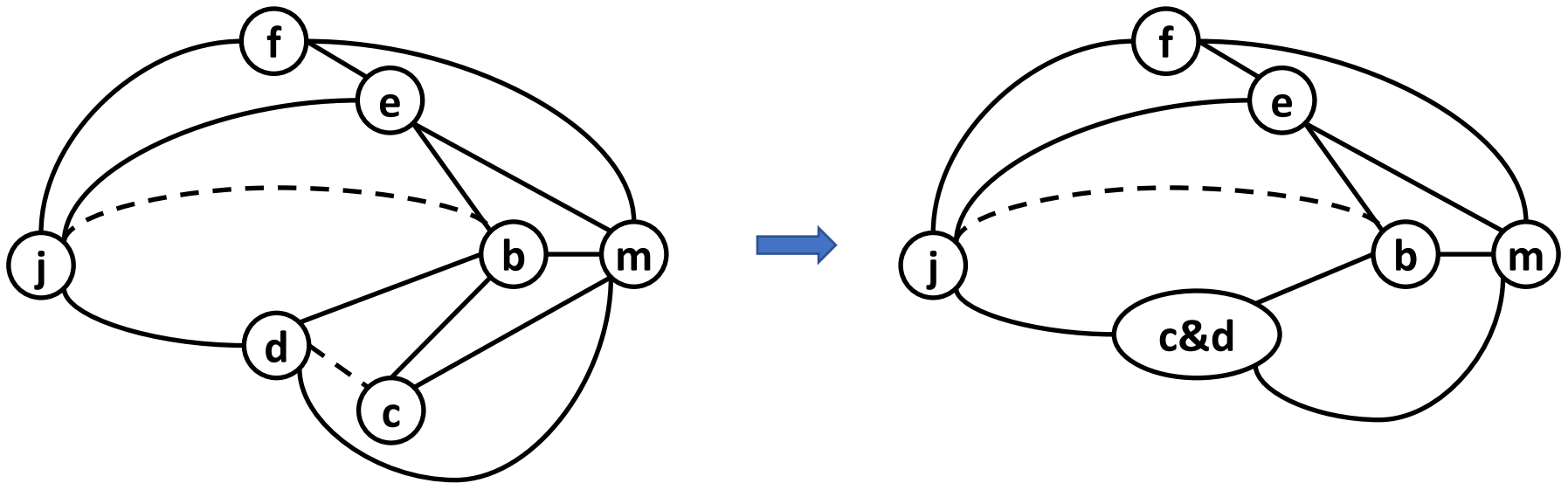
Example

- $K = 4$
- Nodes **d**, **c**, **b** and **j** are the only move-related nodes.
- The initial work-list used in the simplify phase must contain only non-move-related nodes.
 - **f**, **g**, **h**
- After the removal of **g**, **h**, **k**



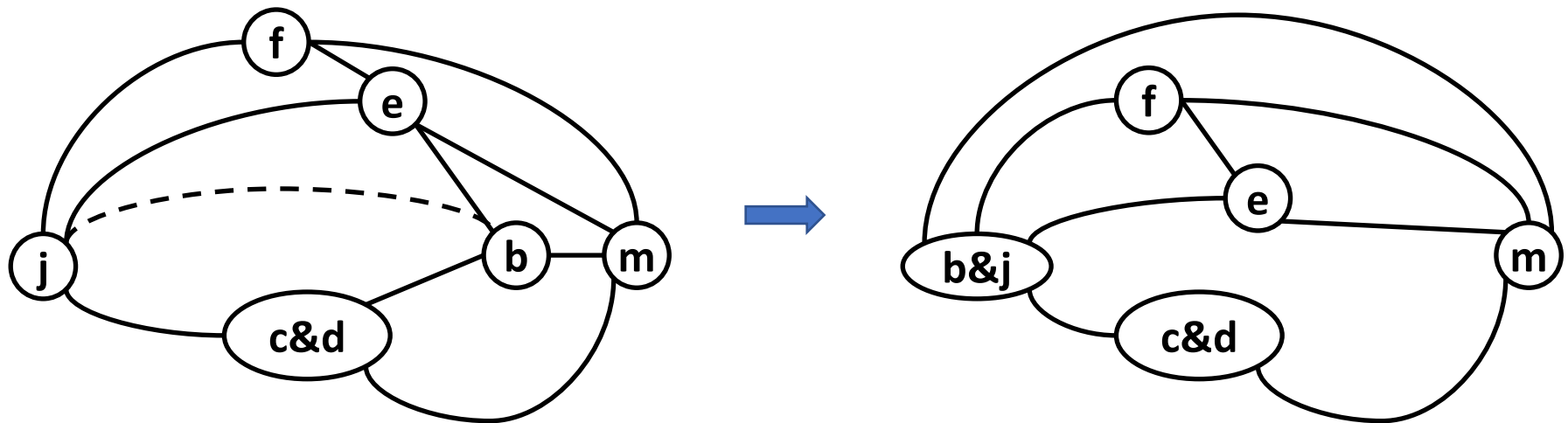
Example

- If we invoke a round of coalescing at this point, we discover that **c** and **d** are indeed coalesceable.
 - Why?



Example

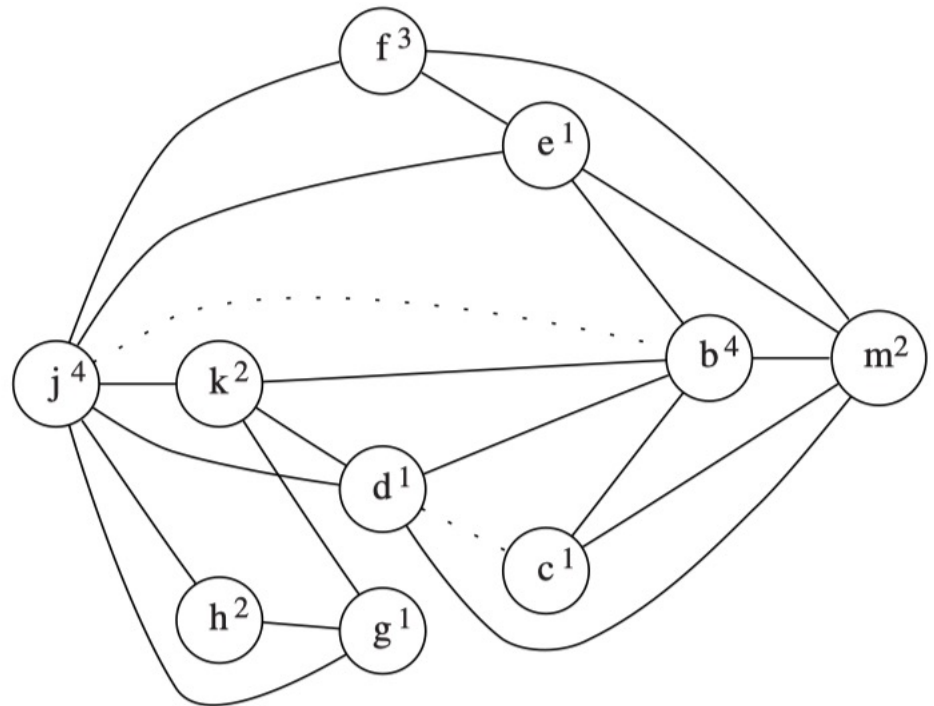
- It is possible to coalesce **b** and **j**.
- There are no more move-related nodes, and therefore no more coalescing is possible.
- The simplify phase can be invoked one more time to remove all the remaining nodes.



Example

- A possible assignment of colors:
- **j** and **b** are assigned the same register; **c** and **d** are assigned the same register. The corresponding MOVE instructions can be eliminated.

e	1
m	2
f	3
j&b	4
c&d	1
k	2
h	2
g	1
stack	coloring



Outline

- Coloring By Simplification
- Coalescing
- **Precolored Nodes**

Precolored Nodes

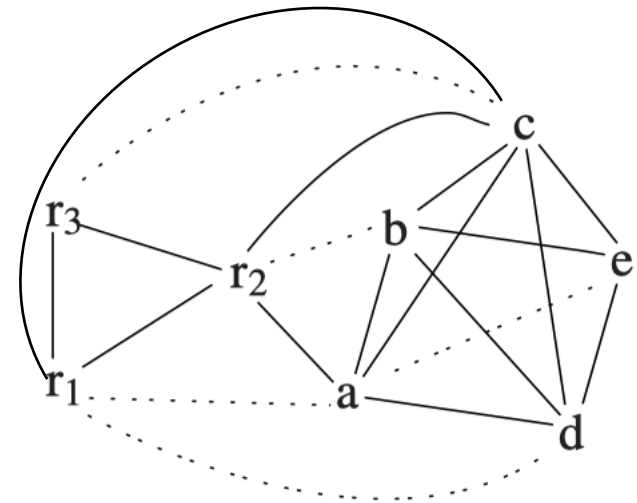
- Some registers are used for special purpose:
 - argument-registers
 - frame pointer
 - return-value-register
 - etc.
- For each of such registers, use the particular temporary that is permanently bound to that register. (E.g., FP)
- Such temporaries are **precolored**.
 - Only one precolored node of each color
 - Precolored nodes all interfere with each other.

Precolored Nodes

- It is common to give an ordinary temporary the same color as a precolored register, as long as they don't interfere.
 - E.g., a standard calling-convention register can be reused inside a procedure as a temporary variable
- We **cannot** simplify a precoloared node.
- We **should not** spill precolored nodes to memory.
 - Machine registers are by definition registers

George Criterion with Pre-colored Nodes

- **George Criterion for Coalescing:** Nodes **a** and **b** can be coalesced if for every neighbor **t** of **a**, either **t** already interferes with **b** or **t** is of insignificant degree ($< K$).
- When applying the George criterion to node pairs involving a pre-colored node, always pick the one that is not pre-colored as **a** to test the rule.
 - E.g., $k=3$, **r3** and **c**
 - If we coalesce **r3** and **c**, we will make a node that should not be colored (i.e., **c**) be colored as **r3**, and cannot spill **c**.



Temporary Copies of Machine Registers

- The coloring algorithm works by calling **simplify**, **coalesce**, and **spill** until only the precolored nodes remain.
- Because precolored nodes do not spill, the front end must be careful to keep their live ranges short:
 - by generating MOVE instructions to move values to and from precolored nodes.
- Suppose r7 is a callee-save register:

Enter:	def(r7)
	⋮
Exit:	use(r7)

Enter:	def(r7)
	$t_{231} \leftarrow r7$
	⋮
	$r7 \leftarrow t_{231}$
Exit:	use(r7)

- If there is register pressure (a high demand for registers) in this function, **t231** will spill; otherwise **t231** will be coalesced with **r7** and the MOVE instructions will be eliminated.

Caller-Save and Callee-Save Registers

```
foo (){  
  t = ...  
  ... = ... t ...  
  s = ...  
  f()  
  g()  
  ... = ... s ...  
}
```

A local variable or compiler temporary that is not live across any procedure call should usually be allocated to a caller-save register

Any variable that is live across several procedure calls should be kept in a callee-save-register

- If a variable **x** is live across a procedure call,
 - then it interferes with all the caller-save (precolored) registers
 - *and* it interferes with all the new temporaries created for callee-save registers (e.g., t231)
 - a spill will occur
 - Using the common spill-cost heuristic that spills a node with high degree but few uses
 - which variable will be spilled first? x or t231?

Example with Precolored Nodes

```
int f(int a, int b) {  
    int d = 0;  
    int e = a;  
    do {  
        d = d+b;  
        e = e-1;  
    } while (e>0);  
    return d;  
}
```

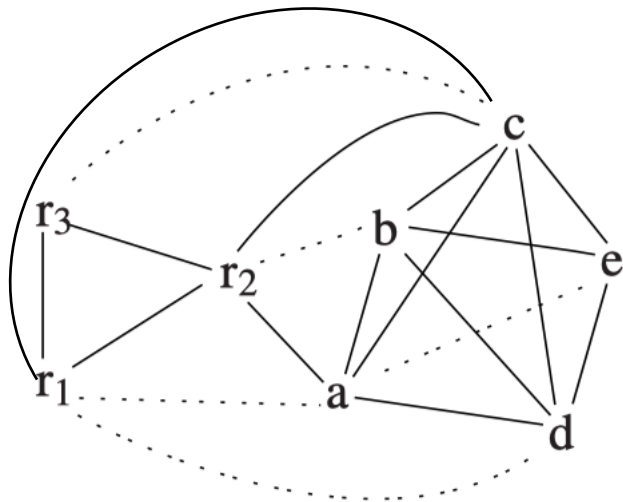


```
enter:  c ← r3  
        a ← r1  
        b ← r2  
        d ← 0  
        e ← a  
loop:   d ← d + b  
        e ← e - 1  
        if (e > 0) goto loop  
        r1 ← d  
        r3 ← c  
return  (r1, r3 live out)
```

For a machine with 3 registers:

- **r1** and **r2** are caller-save
- **r3** is callee save

Example with Precolored Nodes (K=3)

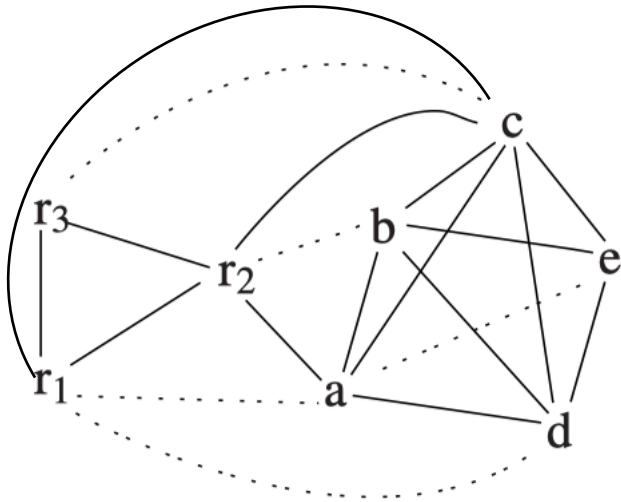


Interference Graph

```
enter:  c ← r3
        a ← r1
        b ← r2
        d ← 0
        e ← a
loop:   d ← d + b
        e ← e - 1
        if (e > 0) goto loop
        r1 ← d
        r3 ← c
        return (r1, r3 live out)
```

- No opportunity for **simplify** and **freeze**
 - All the non-precolored nodes have degree $\geq K$
- **Coalesce**:
 - **Briggs**: would produce a coalesced node adjacent to $\geq K$ significant-degree nodes.
 - **George**: e.g., for **r3** and **c**, **c** does not meet the criterion
- We must **spill** some node

Example with Precolored Nodes (K=3)

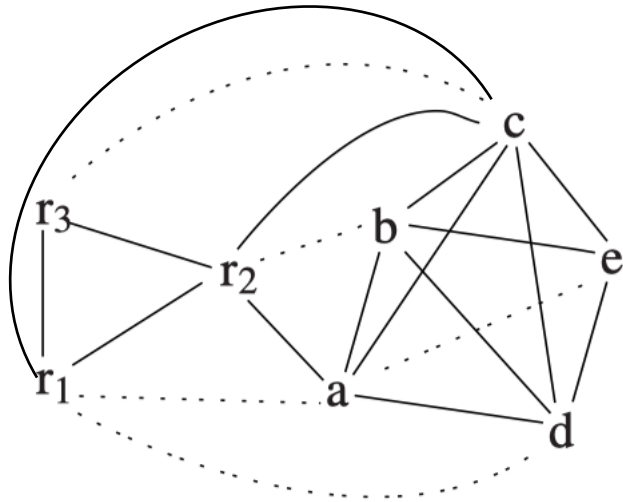


Interference Graph

```
enter:  c ← r3
        a ← r1
        b ← r2
        d ← 0
        e ← a
loop:   d ← d + b
        e ← e - 1
        if (e > 0) goto loop
        r1 ← d
        r3 ← c
return  (r1, r3 live out)
```

- We must **spill** some node
- How to choose the node to **spill**?
- Answer: the node with more degrees, but is rarely used
 - why?

Example with Precolored Nodes (K=3)



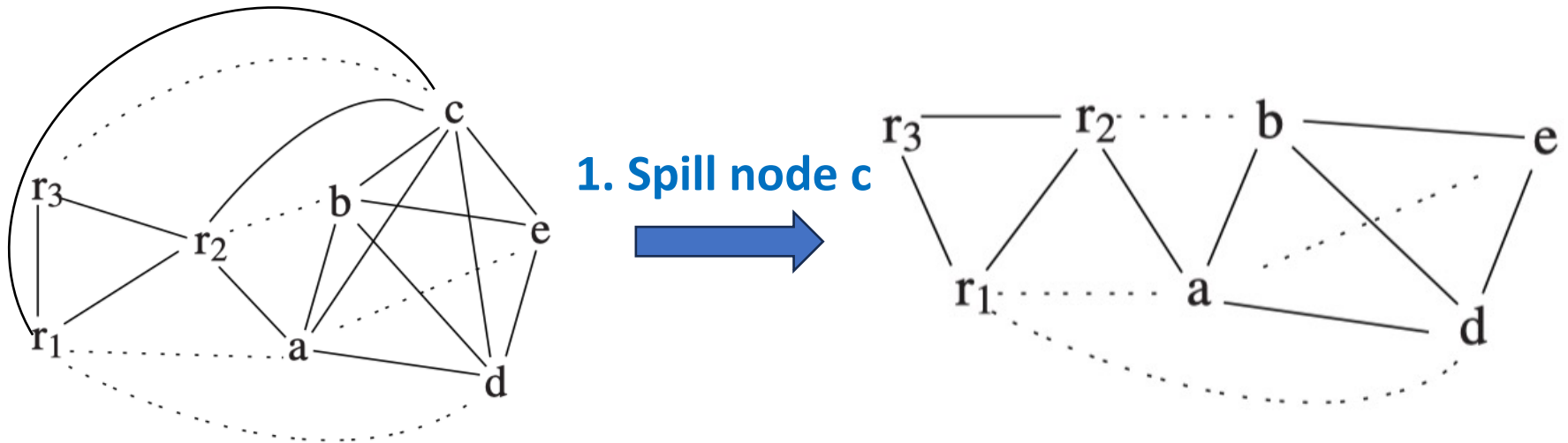
Interference Graph

```

enter:  c ← r3
        a ← r1
        b ← r2
        d ← 0
        e ← a
loop:   d ← d + b
        e ← e - 1
        if (e > 0) goto loop
        r1 ← d
        r3 ← c
return  (r1, r3 live out)
    
```

Node	Uses + Defs outside Loop	Uses + Defs within loop	Degree	Spill Priority
a	(2	+ 10 x 0)	/ 4	= 0.50
b	(1	+ 10 x 1)	/ 4	= 2.75
c	(2	+ 10 x 0)	/ 6	= 0.33
d	(2	+ 10 x 2)	/ 4	= 5.50
e	(1	+ 10 x 3)	/ 3	= 10.33

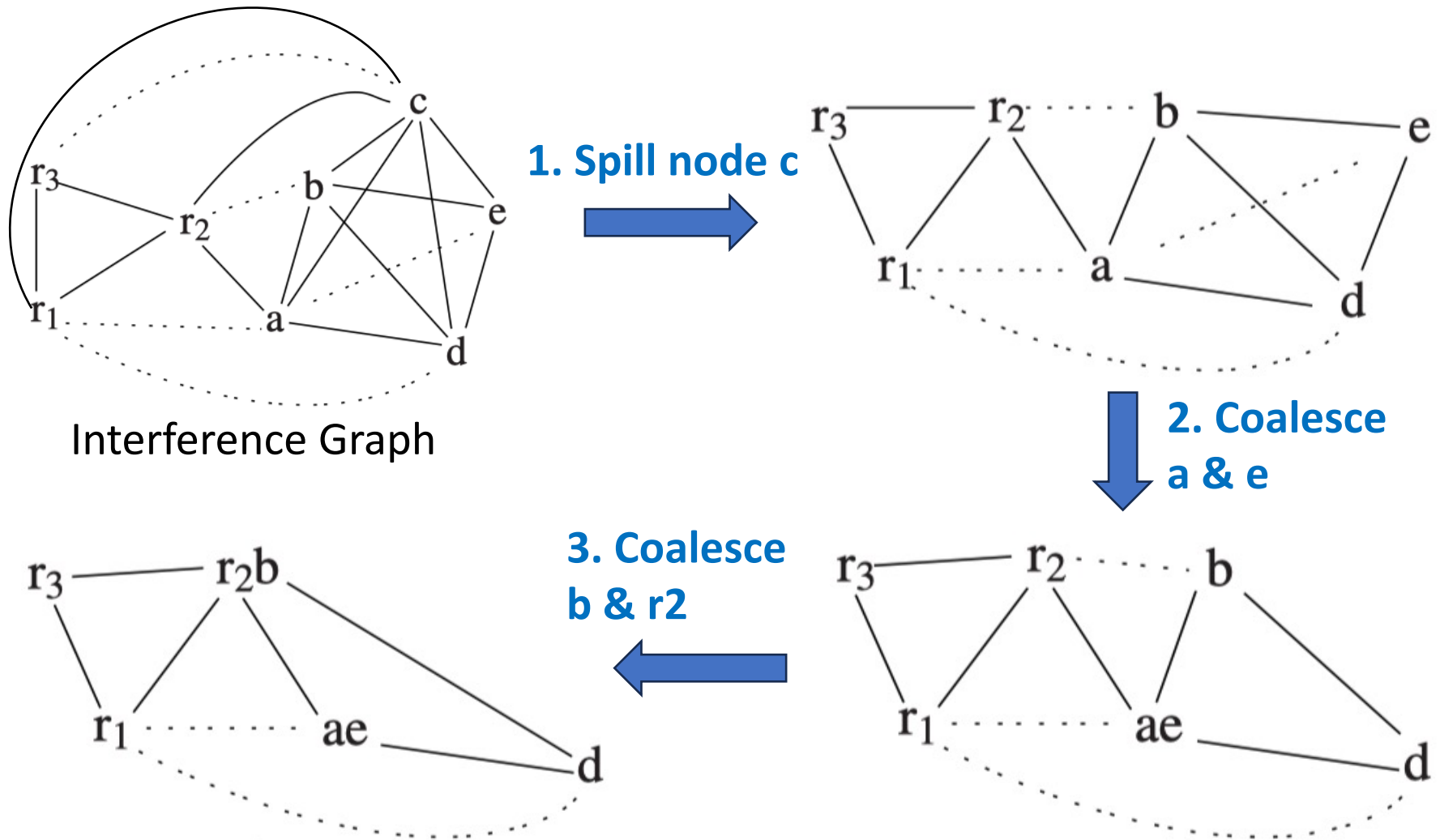
Example with Precolored Nodes (K=3)



Interference Graph

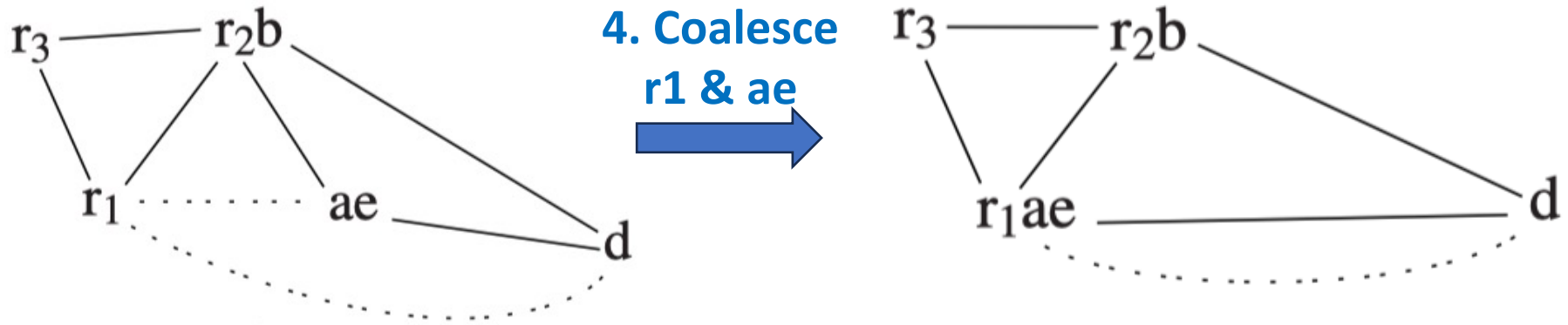
- Spill node c
- No simplify is possible
 - All non-precolored nodes are move-related

Example with Precolored Nodes (K=3)



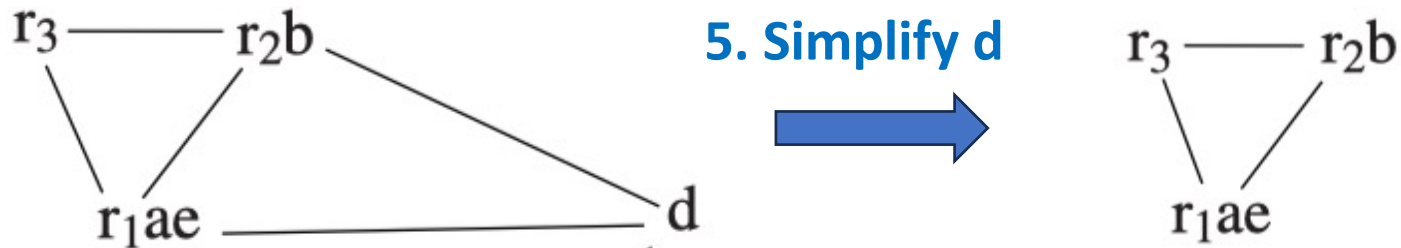
- Perform coalescing

Example with Precolored Nodes (K=3)



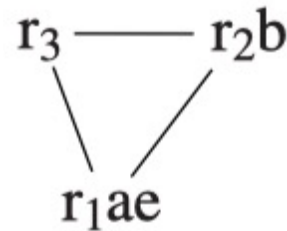
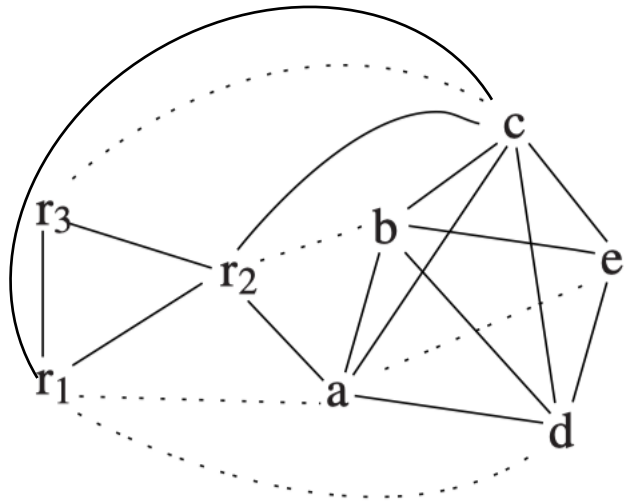
- Perform coalescing
- Now, can we coalesce **r1ae** and **d**?
 - No, **r1ae** interferes with **d**
- The move between **r1ae** and **d** is **constrained**
 - We remove it from further considerations
 - **d** is no longer treated as move-related

Example with Precolored Nodes (K=3)



- We must simplify d
- Now, only precolored nodes.

Example with Precolored Nodes (K=3)

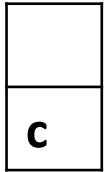
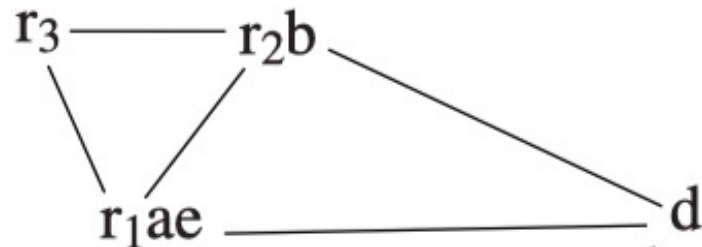
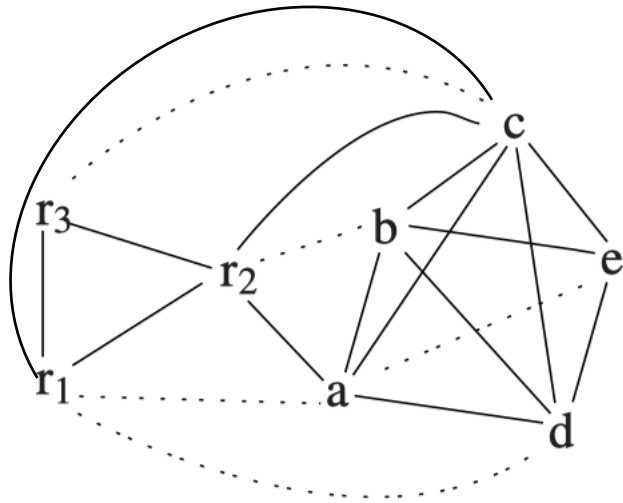


d
c

6. Select

- Pop nodes from the stack and assign color to them:
 - Nodes **a**, **b**, **e** have already been assigned colors by coalescing
 - Pick **d**, assign color **r3**

Example with Precolored Nodes (K=3)



6. Select

- Pop nodes from the stack and assign color to them:
 - Nodes **a**, **b**, **e** have already been assigned colors by coalescing
 - Pick **d**, assign color **r3**
 - Pop **c**: c turns into an **actual spill** (**Why?**)

Example with Precolored Nodes (K=3)

7. Rewrite

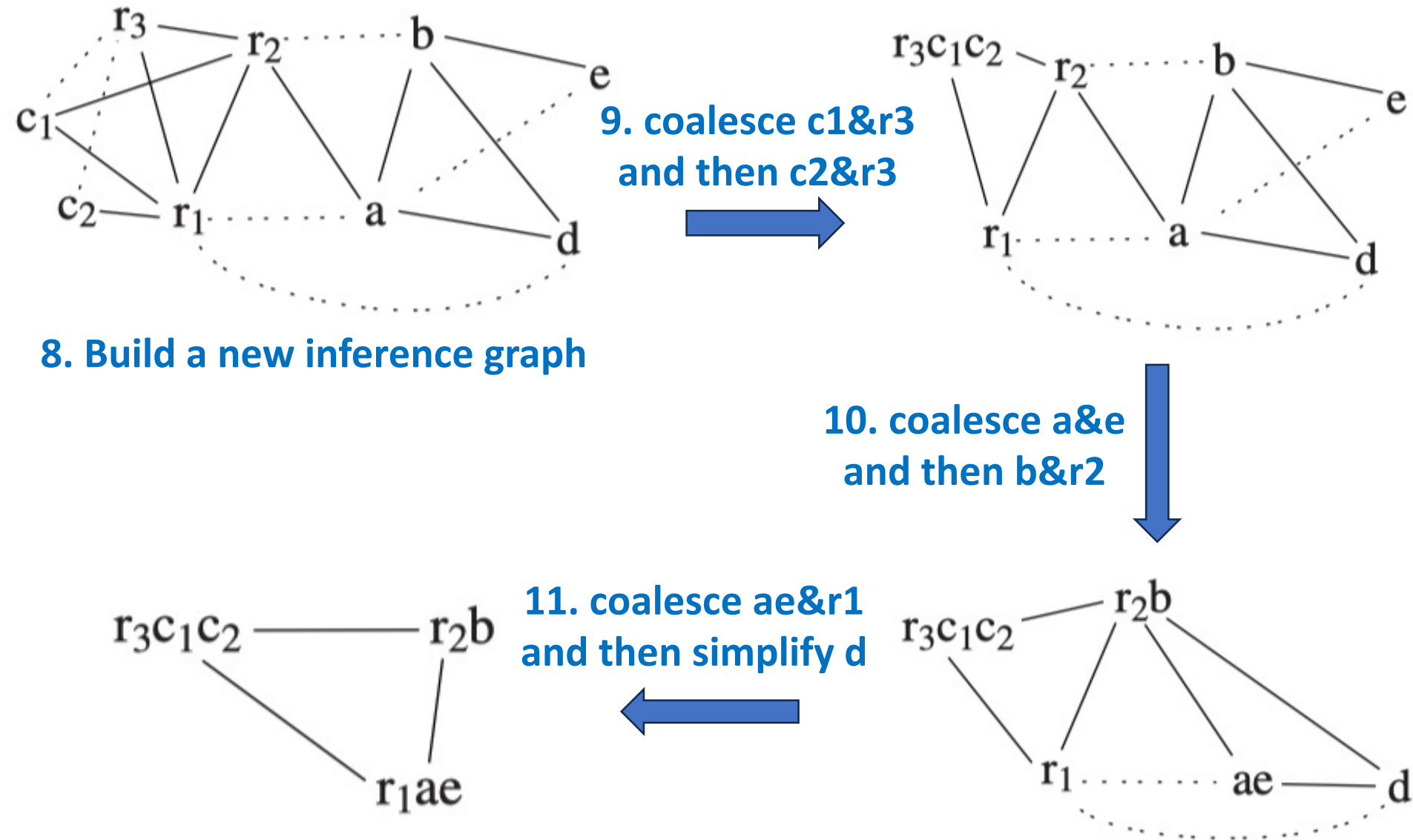
- There is spilling, we must rewrite the program to include spill instructions.
 - Before each **use** -> fetch
 - After each **def** -> store

```
enter:  c ← r3
        a ← r1
        b ← r2
        d ← 0
        e ← a
loop:   d ← d + b
        e ← e - 1
        if (e > 0) goto loop
        r1 ← d
        r3 ← c
        return
```

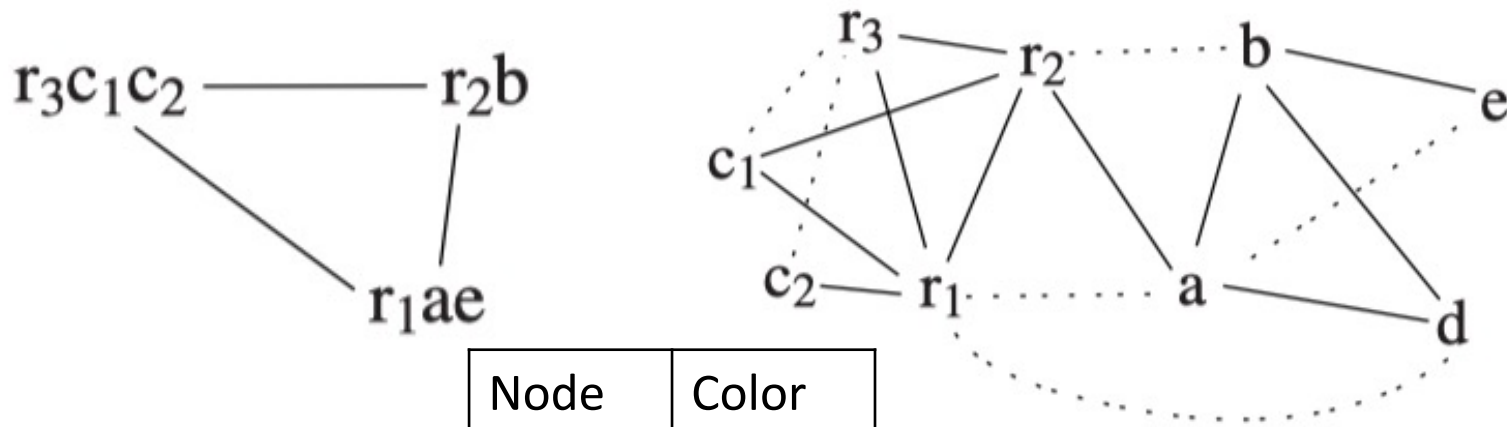
rewrite


```
enter:  c1 ← r3
        M[cloc] ← c1
        a ← r1
        b ← r2
        d ← 0
        e ← a
loop:   d ← d + b
        e ← e - 1
        if (e > 0) goto loop
        r1 ← d
        c2 ← M[cloc]
        r3 ← c2
        return
```

Example with Precolored Nodes (K=3)



Example with Precolored Nodes (K=3)



Node	Color
a	r1
b	r2
c1	r3
c2	r3
d	r3
e	r1

12. Select

- Popping from the stack and select color **r3** for **d**
- all other nodes were coalesced or precolored

Example with Precolored Nodes (K=3)

13. Rewrite the program using the register assignment

```
enter:  c1 ← r3
        M[cloc] ← c1
        a ← r1
        b ← r2
        d ← 0
        e ← a
loop:   d ← d + b
        e ← e - 1
        if (e > 0) goto loop
        r1 ← d
        c2 ← M[cloc]
        r3 ← c2
        return
```

Node	Color
a	r1
b	r2
c1	r3
c2	r3
d	r3
e	r1



```
enter:  r3 ← r3
        M[cloc] ← r3
        r1 ← r1
        r2 ← r2
        r3 ← 0
        r1 ← r1
loop:   r3 ← r3 + r2
        r1 ← r1 - 1
        if (r1 > 0) goto loop
        r1 ← r3
        r3 ← M[cloc]
        r3 ← r3
        return
```

Example with Precolored Nodes (K=3)

14. delete any move instruction whose source and destination are the same:

```
enter:  r3 ← r3
        M[cloc] ← c1
        r1 ← r1
        r2 ← r2
        r3 ← 0
        r1 ← r1
loop:   r3 ← r3 + r2
        r1 ← r1 - 1
        if (r1 > 0) goto loop
        r1 ← r3
        r3 ← M[cloc]
        r3 ← r3
        return
```



```
enter:  M[cloc] ← c1
        r3 ← 0
loop:   r3 ← r3 + r2
        r1 ← r1 - 1
        if (r1 > 0) goto loop
        r1 ← r3
        r3 ← M[cloc]
        return
```

Summary

- Register allocation by graph coloring
 - Build
 - Simplify
 - Coalesce
 - Freeze
 - Spill
 - Select
- Precolored nodes

Homework

- **11.1, 11.3**